

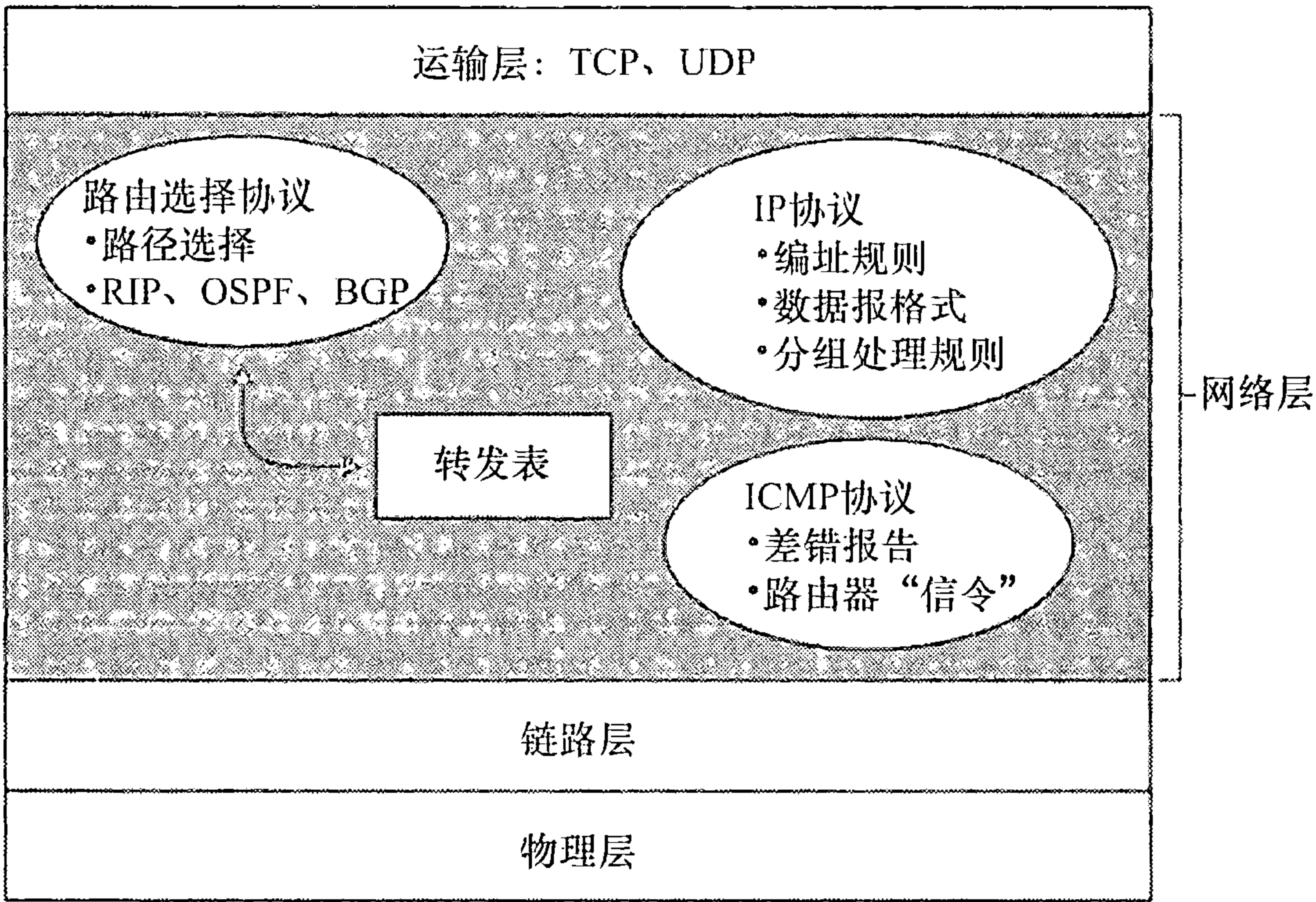


图 4-12 因特网网络层的内部视图

4.4.1 数据报格式

前面讲过网络层分组被称为数据报。我们以概述 IPv4 数据报的语法和语义开始对 IP 的学习。你也许认为没有什么比一个分组的比特的语法和语义更加枯燥无味的了。无论如何，数据报在因特网中起着重要作用，每个网络行业的学生和专业人员都需要理解它、吸收它并掌握它。IPv4 数据报格式如图 4-13 所示。

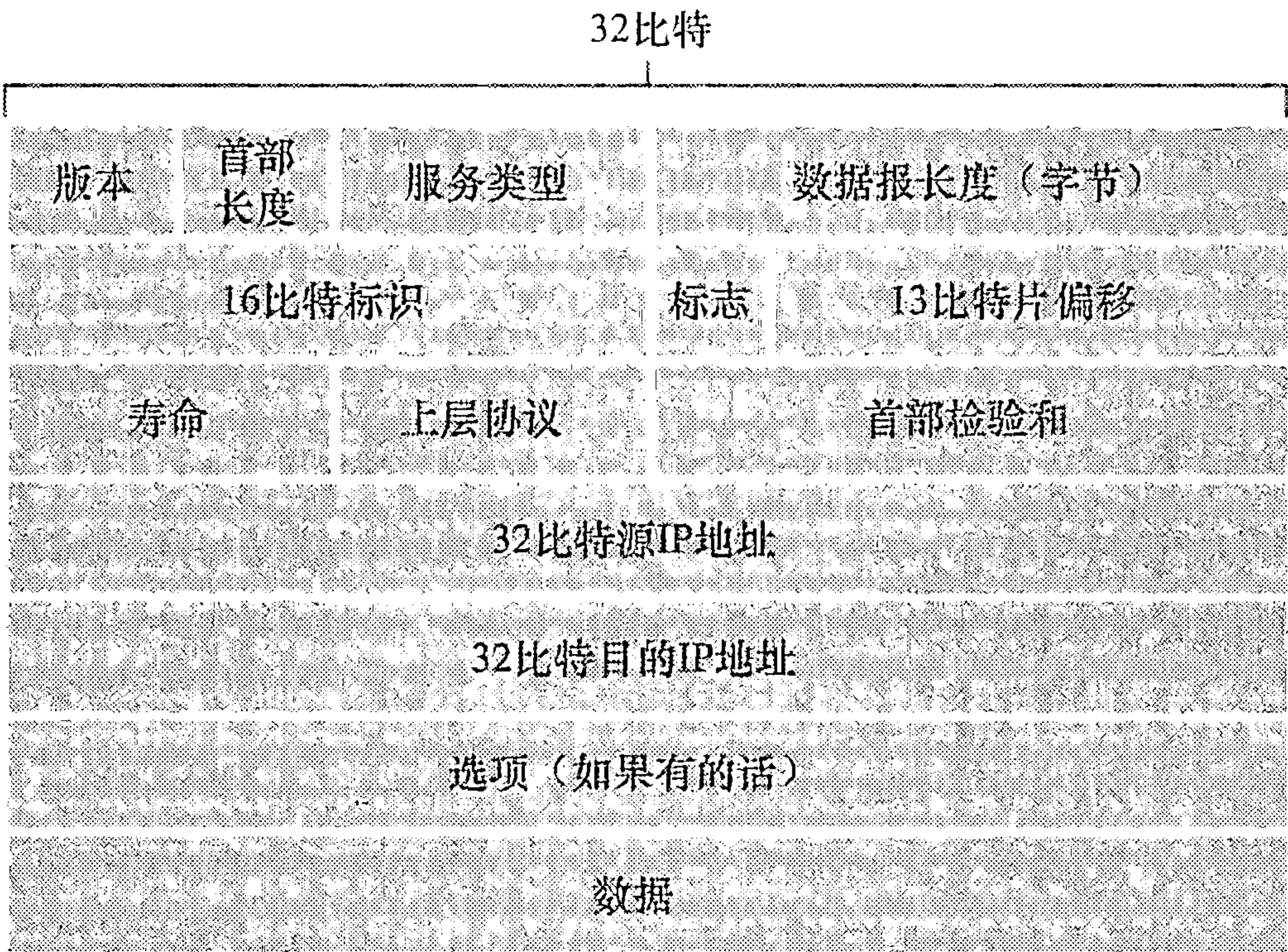


图 4-13 IPv4 数据报格式

IPv4 数据报中的关键字段如下：

- 版本（号）。这 4 比特规定了数据报的 IP 协议版本。通过查看版本号，路由器能够确定如何解释 IP 数据报的剩余部分。不同的 IP 版本使用不同的数据报格式。目前版本的 IP（即 IPv4）的数据报格式如图 4-13 所示。新版本的 IP（IPv6）的数据报格式将在本节后面讨论。

- ◉ 首部长度的。因为一个 IPv4 数据报可包含一些可变数量的选项（这些选项包括在 IPv4 数据报首部中），故需要用这 4 比特来确定 IP 数据报中数据部分实际从哪里开始。大多数 IP 数据报不包含选项，所以一般的 IP 数据报具有 20 字节的首部。
- ◉ 服务类型。服务类型（TOS）比特包含在 IPv4 首部中，以便使不同类型的 IP 数据报（例如，一些特别要求低时延、高吞吐量或可靠性的数据报）能相互区别开来。例如，将实时数据报（如用于 IP 电话应用）与非实时流量（如 FTP）区分开也许是有用的。提供特定等级的服务是一个由路由器管理员决定的策略问题。我们将在第 7 章详细讨论区分服务主题。
- ◉ 数据报长度。这是 IP 数据报的总长度（首部加上数据），以字节计。因为该字段长为 16 比特，所以 IP 数据报的理论最大长度为 65 535 字节。然而，数据报很少有超过 1500 字节的。
- ◉ 标识、标志、片偏移。这三个字段与所谓 IP 分片有关，这是一个我们将很快要深入考虑的一个问题。有趣的是，新版本的 IP（即 IPv6）不允许在路由器上对分组分片。
- ◉ 寿命。寿命（Time-To-Live, TTL）字段用来确保数据报不会永远（如由于长时间的路由选择环路）在网络中循环。每当数据报由一台路由器处理时，该字段的值减 1。若 TTL 字段减为 0，则该数据报必须丢弃。
- ◉ 协议。该字段仅在一个 IP 数据报到达其最终目的地才会有用。该字段值指示了 IP 数据报的数据部分应交给哪个特定的运输层协议。例如，值为 6 表明数据部分要交给 TCP，而值为 17 表明数据要交给 UDP。对于所有可能值的列表，参见 [IANA Protocol Numbers 2012]。注意在 IP 数据报中的协议号所起的作用，类似于运输层报文段中端口号字段所起的作用。协议号是将网络层与运输层绑定到一起的粘合剂，而端口号是将运输层和应用层绑定到一起的粘合剂。我们将在第 5 章看到，链路层帧也有一个特殊字段用于将链路层与网络层绑定到一起。
- ◉ 首部检验和。首部检验和用于帮助路由器检测收到的 IP 数据报中的比特错误。首部检验和是这样计算的：将首部中的每 2 个字节当作一个数，用反码运算对这些数求和。如在 3.3 节讨论的那样，该和的反码（被称为因特网检验和）存放在检验和字段中。路由器要对每个收到的 IP 数据报计算其首部检验和，如果数据报首部中携带的检验和与计算得到的检验和不一致，则检测出是个差错。路由器一般会丢弃检测出错误的数据报。注意到在每台路由器上必须重新计算检验和并再次存放到原处，因为 TTL 字段以及可能的选项字段会改变。关于计算因特网检验和的快速算法的有趣讨论参见 [RFC 1071]。此时，一个经常问到的问题是：为什么 TCP/IP 在运输层与网络层都执行差错检测？这种重复检测有几种原因。首先，注意到在 IP 层只对 IP 首部计算了检验和，而 TCP/UDP 检验和是对整个 TCP/UDP 报文段进行的。其次，TCP/UDP 与 IP 不一定都必须属于同一个协议栈。原则上 TCP 能运行在一个不同的协议（如 ATM）上，而 IP 能够携带不一定要传递给 TCP/UDP 的数据。
- ◉ 源和目的 IP 地址。当某源生成一个数据报时，它在源 IP 字段中插入它的 IP 地址，在目的 IP 地址字段中插入其最终目的地的地址。通常源主机通过 DNS 查找来决定目的地址，如在第 2 章中讨论的那样。我们将在 4.4.2 节中详细讨论了 IP 编址。

- 选项。选项字段允许 IP 首部被扩展。首部选项意味着很少使用，因此决定对每个数据报首部不包括选项字段中的信息，这样能够节约开销。然而，选项的可能存在的确是件复杂的事，因为数据报头长度可变，故不能预先确定数据字段从何处开始。而且还因为有些数据报要求处理选项，而有些数据报则不要求，故导致一台路由器处理一个 IP 数据报所需的时间变化很大。这些考虑对于高性能路由器和主机上的 IP 处理来说特别重要。由于这样或那样的原因，在 IPv6 首部中已去掉了 IP 选项，如 4.4.4 节中讨论的那样。
- 数据（有效载荷）。我们来看看最后的也是最重要的字段，这是数据报存在的首要理由！在大多数情况下，IP 数据报中的数据字段包含要交付给目的地的运输层报文段（TCP 或 UDP）。然而，该数据字段也可承载其他类型的数据，如 ICMP 报文（在 4.4.3 节中讨论）。

注意到一个 IP 数据报有总长为 20 字节的首部（假设无选项）。如果数据报承载一个 TCP 报文段，则每个（无分片的）数据报共承载了总长 40 字节的首部（20 字节的 IP 首部加上 20 字节的 TCP 首部）以及应用层报文。

IP 数据报分片

在第 5 章中我们将看到，并不是所有链路层协议都能承载相同长度的网络层分组。有的协议能承载大数据报，而有的协议只能承载小分组。例如，以太网帧能够承载不超过 1500 字节的数据，而某些广域网链路的帧可承载不超过 576 字节的数据。一个链路层帧能承载的最大数据量叫做最大传送单元（Maximum Transmission Unit, MTU）。因为每个 IP 数据报封装在链路层帧中从一台路由器传输到下一台路由器，故链路层协议的 MTU 严格地限制着 IP 数据报的长度。对 IP 数据报长度具有严格限制并不是主要问题。问题在于在发送方与目的地路径上的每段链路可能使用不同的链路层协议，且每种协议可能具有不同的 MTU。

为了更好地理解这一转发问题，想象你是一台互联几条链路的路由器，且每条链路运行具有不同 MTU 的链路层协议。假定你从某条链路收到一个 IP 数据报，通过检查转发表确定出链路，并且该出链路的 MTU 比该 IP 数据报的长度要小。此时你会感到慌乱，如何将这个过大的 IP 分组压缩进链路层帧的有效载荷字段呢？解决该问题的方法是将 IP 数据报中的数据分片成两个或更多个较小的 IP 数据报，用单独的链路层帧封装这些较小的 IP 数据报；然后向输出链路上发送这些帧。每个这些较小的数据报都称为片（fragment）。

片在其到达目的地运输层以前需要重新组装。实际上，TCP 与 UDP 都希望从网络层收到完整的未分片的报文。IPv4 的设计者感到在路由器中重新组装数据报会给协议带来相当大的复杂性并且影响路由器性能。（如果你是一台路由器，你愿意将重新组装报文片放在你必须要做各种各样工作的首位吗？）为坚持网络内核保持简单的原则，IPv4 的设计者决定将数据报的重新组装工作放到端系统中，而不是放到网络路由器中。

当一台目的主机从相同源收到一系列数据报时，它需要确定这些数据报中的某些是否是一些原来较大的数据报的片。如果某些数据报是片的话，则它必须进一步确定何时收到了最后一片，并且如何将这些接收到的片拼接到一起以形成初始的数据报。为了让目的主机执行这些重新组装任务，IPv4 的设计者将标识、标志和片偏移字段放在 IP 数据报首部中。当生成一个数据报时，发送主机在为该数据报设置源和目的地址的同时再贴上标识

号。发送主机通常将为它发送的每个数据报的标识号加 1。当某路由器需要对一个数据报分片时，形成的每个数据报（即片）具有初始数据报的源地址、目的地址与标识号。当目的地从同一发送主机收到一系列数据报时，它能够检查数据报的标识号以确定哪些数据报实际上是同一较大数据报的片。由于 IP 是一种不可靠的服务，一个或多个片可能永远到达不了目的地。因为这种原因，为了让目的主机绝对地相信它已收到了初始数据报的最后一个片，最后一个片的标志比特被设为 0，而所有其他片的标志比特被设为 1。另外，为了让目的主机确定是否丢失了一个片（且能按正确的顺序重新组装片），使用偏移字段指定该片应放在初始 IP 数据报的哪个位置。

图 4-14 图示了一个例子。一个 4000 字节的数据报（20 字节 IP 首部加上 3980 字节 IP 有效载荷）到达一台路由器，且必须被转发到一条 MTU 为 1500 字节的链路上。这就意味着初始数据报中 3980 字节数据必须被分配为 3 个独立的片（其中的每个片也是一个 IP 数据报）。假定初始数据报贴上的标识号为 777。三个片的特点如表 4-2 所示。表 4-2 中的值反映了除了最后一片的所有初始有效载荷数据的数量应当是 8 字节的倍数，并且偏移值应当被规定以 8 字节块为单位。

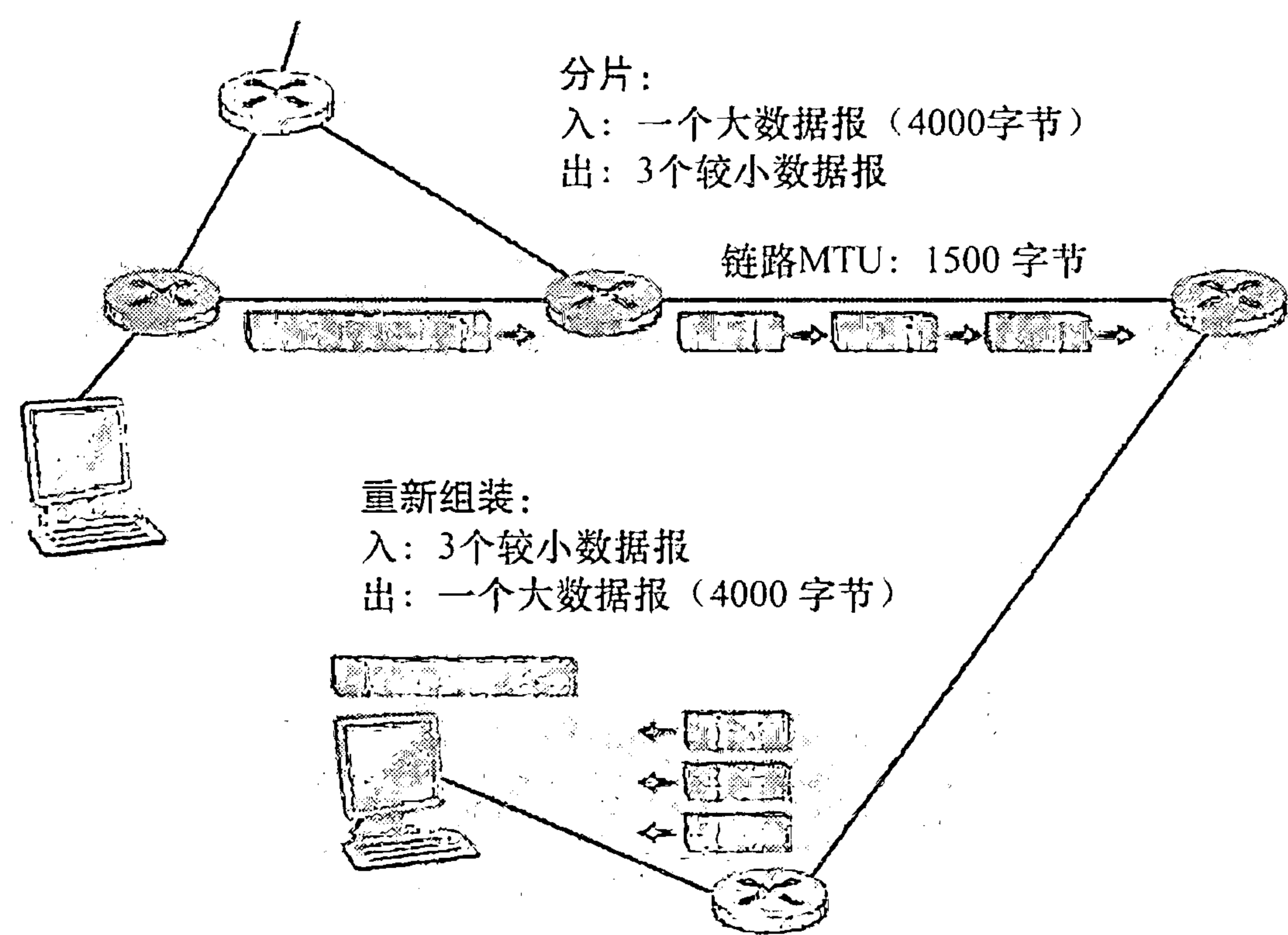


图 4-14 IP 分片与重新组装

表 4-2 IP 片

片	字节	ID	偏移	标志
第 1 片	IP 数据报的数据字段中的 1480 字节	identification = 777	offset = 0（表示插入的数据开始于字节 0）	flag = 1（表示后面还有）
第 2 片	1480 字节数据	identification = 777	offset = 185（表示插入的数据开始于字节 1480。注意 185 × 8 = 1480）	flag = 1（表示后面还有）
第 3 片	1020 字节数据（= 3980 - 1480 - 1480）	identification = 777	offset = 370（表示插入的数据开始于字节 2960。注意 370 × 8 = 2960）	flag = 0（表示这是最后一个片）

在目的地，数据报的有效载荷仅当在 IP 层已完全重构为初始 IP 数据报时，才被传递给目的地运输层。如果一个或多个片没有到达目的地，则该不完整的数据报被丢弃且不会

交给运输层。但是，如我们在前一章知道的那样，若在运输层正使用着 TCP，则 TCP 将通过让源以初始数据报来重传数据，以恢复这次丢包。

我们刚刚学习了 IP 分片在将许多完全不同的链路层技术粘合起来的过程中起到了重要作用。但是分片也是有开销的。首先，它使路由器和端系统更为复杂。其次，分片能够被用于生成致命的 DoS 攻击，由此攻击者发送了一系列古怪的、无法预期的片。Jolt2 攻击就是一个经典的例子，其中某攻击者向目标主机发送了小片的流，这些片中没有一个片的偏移量是 0。当目标试图从这些不良分组中重建数据报时，可能会崩溃。另一类行为是发送交迭的 IP 片，即这些片的偏移量值被设置得不能够适当地排列起来。易受攻击的操作系统由于对于交迭的片不知道该如何应对，可能会崩溃 [Skoudis 2006]。如我们将在本节结束看到的那样，IP 协议的新版本 IPv6 从根本上废止了分片，从而简化了 IP 分组的处理，并使得 IP 不太容易受到攻击。

在本书的 Web 站点，我们提供了一个 Java 小程序来产生片。你提供入数据报长度、MTU 和入数据报标识，它就自动为你产生片。参见 <http://www.awl.com/kurose-ross>。

4.4.2 IPv4 编址

我们现在将注意力转向 IPv4 编址。尽管你可能认为编址是相当直接的主题，但我们希望通过本章的学习，你能认识到因特网编址不仅是一个丰富多彩、微妙和有趣的主题，而且也是一个对因特网极为重要的主题。[3Com Addressing 2012] 和 [Stewart 1999] 的第 1 章都是介绍 IPv4 编址的优秀读物。

然而，在讨论 IP 编址之前，我们需要简述一下主机与路由器连入网络的方法。一台主机通常只有一条链路连接到网络；当主机中的 IP 想发送一个数据报时，它就在该链路上发送。主机与物理链路之间的边界叫做接口（interface）。现在考虑一台路由器及其接口。因为路由器的任务是从链路上接收数据报并从某些其他链路转发出去，路由器必须拥有两条或更多条链路与它连接。路由器与它的任意一条链路之间的边界也叫做接口。一台路由器因此有多个接口，每个接口有其链路。因为每台主机与路由器都能发送和接收 IP 数据报，IP 要求每台主机和路由器接口拥有自己的 IP 地址。因此，一个 IP 地址技术上是与一个接口相关联的，而不是与包括该接口的主机或路由器相关联的。

每个 IP 地址长度为 32 比特（等价为 4 字节），因此总共有 2^{32} 个可能的 IP 地址。由于 2^{10} 近似地表示 10^3 ，故容易看出约有 40 亿个可能的 IP 地址。这些地址一般按所谓点分十进制记法（dotted-decimal notation）书写，即地址中的每个字节用它的十进制形式书写，各字节间以句号（点）隔开。例如，考虑 IP 地址 193.32.216.9，193 是该地址第一个 8 比特的十进制等价数，32 是该地址第二个 8 比特的十进制等价数，依次类推。因此，地址 193.32.216.9 的二进制记法是：

11000001 00100000 11011000 00001001

在全球因特网中的每台主机和路由器上的每个接口，必须有一个全球唯一的 IP 地址（在 NAT 后面的接口除外，本节结尾将讨论）。然而，这些地址不能随意地自由选择。一个接口的 IP 地址的一部分需要由其连接的子网来决定。

图 4-15 提供了一个 IP 编址与接口的例子。在该图中，一台路由器（具有 3 个接口）用于互联 7 台主机。仔细观察分配给主机和路由器接口的 IP 地址，有几点需要注意。图 4-15 中左上侧部分的 3 台主机以及它们连接的路由器接口，都有一个形如 223.1.1.xxx

的 IP 地址。这就是说，在它们的 IP 地址中，最左侧的 24 比特是相同的。这 4 个接口也通过一个并不包含路由器的网络互联起来。例如，该网络能够是一个以太网 LAN，在此情况下，这些接口将通过一台以太网交换机互联（第 5 章中讨论），或者通过一个无线接入点互联（第 6 章中讨论）。我们此时将这种无路由器连接这些主机的网络表示为一朵云，在第 5、6 章中再深入这些网络的内部。

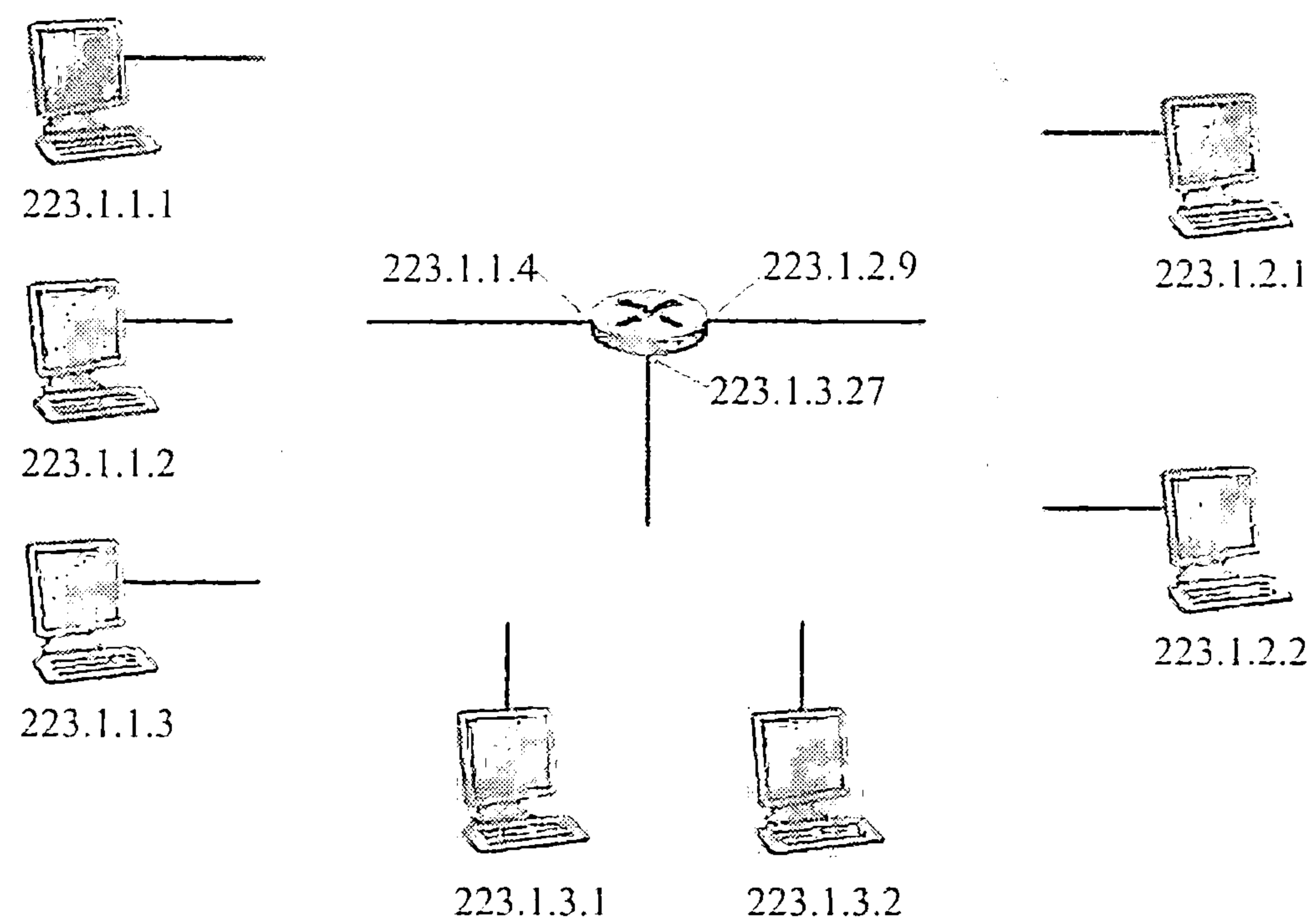


图 4-15 接口地址和子网

用 IP 的术语来说，互联这 3 个主机接口与 1 个路由器接口的网络形成一个子网（subnet）[RFC 950]。（在因特网文献中，子网也称为 IP 网络或直接称为网络。）IP 编址为这个子网分配一个地址：223.1.1.0/24，其中的/24 记法，有时称为子网掩码（network mask），指示了 32 比特中的最左侧 24 比特定义了子网地址。因此子网 223.1.1.0/24 是由 3 台主机接口（223.1.1.1、223.1.1.2 和 223.1.1.3）和 1 个路由器接口（223.1.1.4）组成。任何其他要连到 223.1.1.0/24 网络的主机都要求其地址具有 223.1.1.xxx 的形式。图 4-15 中显示了另外两个网络：223.1.2.0/24 网络与 223.1.3.0/24 子网。图 4-16 图示了在图 4-15 中存在的 3 个 IP 子网。

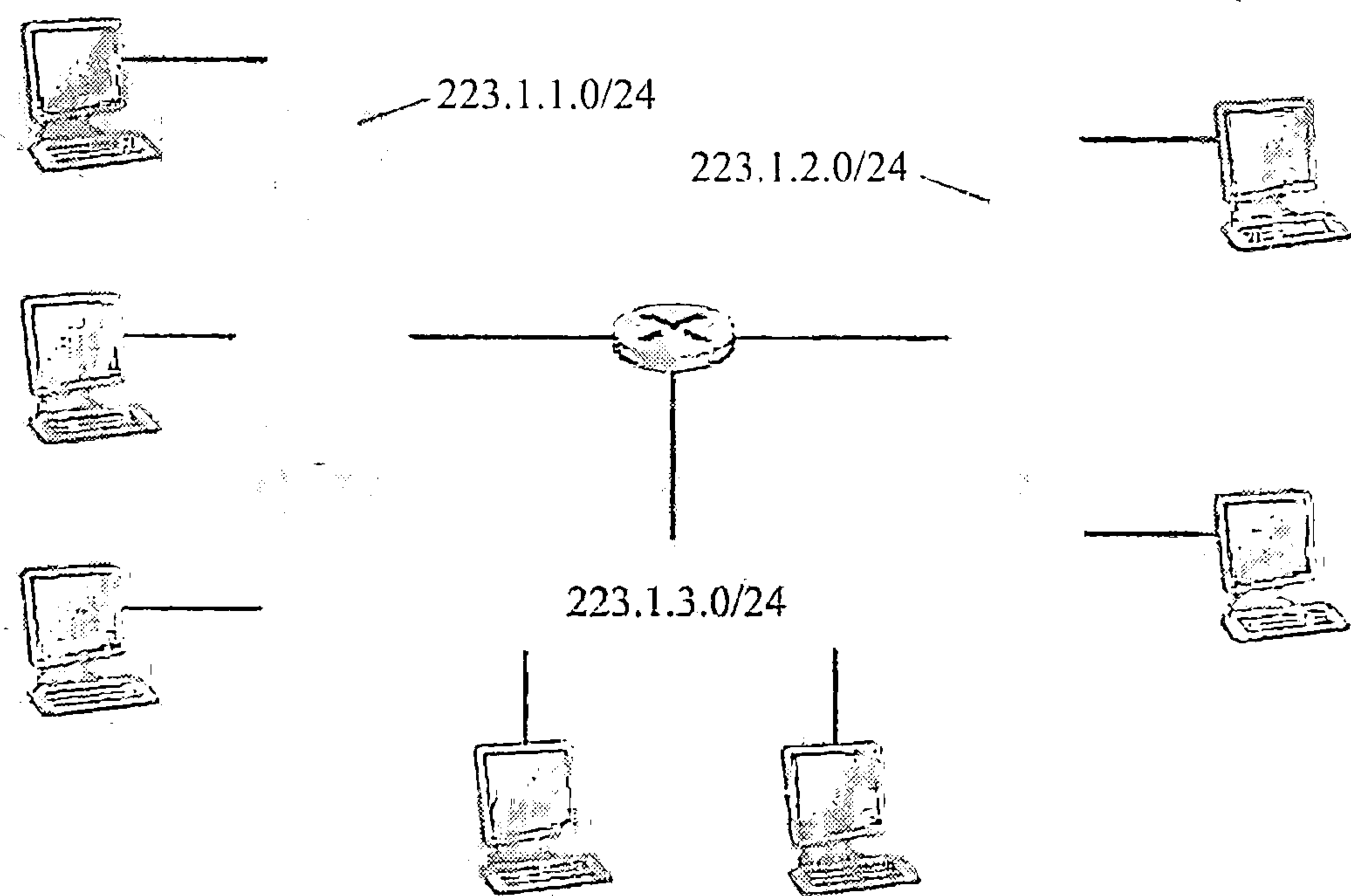


图 4-16 子网地址

一个子网的 IP 定义并不局限于连接多台主机到一个路由器接口的以太网段。为了搞清其中的道理，可考虑图 4-17，图中显示了 3 台通过点对点链路彼此互联的路由器。每台路由器有 3 个接口，每条点对点链路使用一个，一个用于直接将路由器连接到一对主机的广播链路。这里出现了几个子网呢？3 个子网 223.1.1.0/24、223.1.2.0/24 和 223.1.3.0/24 类似于我们在图 4-15 中遇到的子网。但注意到在本例中还有其他 3 个子网：一个子网是 223.1.9.0/24，用于连接路由器 R1 与 R2 的接口；另外一个子网是 223.1.8.0/24，用于连接路由器 R2 与 R3 的接口；第三个子网是 223.1.7.0/24，用于连接路由器 R3 与 R1 的接口。对于一个路由器和主机的通用互联系统，我们能够使用下列有效方法定义系统中的子网：

为了确定子网，分开主机和路由器的每个接口，产生几个隔离的网络岛，使用接口端接这些隔离的网络的端点。这些隔离的网络中的每一个都叫做一个子网 (subnet)。

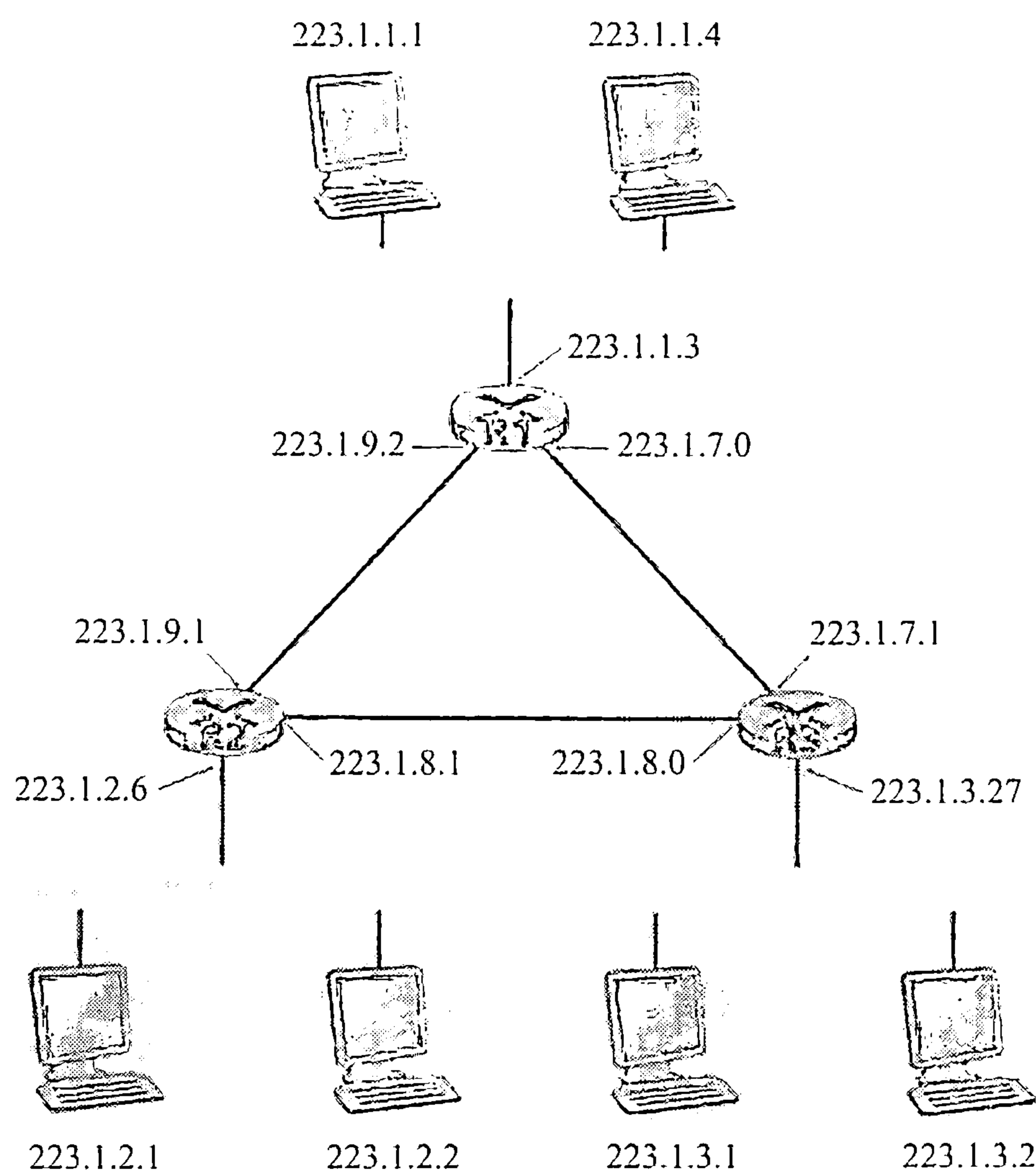


图 4-17 3 台路由器互联 6 个子网

如果我们将该过程用于图 4-17 中的互联系统上，我们会得到 6 个岛或子网。

从上述讨论可见，一个具有多个以太网段和点对点链路的组织（如一个公司或学术机构）将具有多个子网，在给定子网上的所有设备都具有相同的子网地址。原则上，不同的子网能够具有完全不同的子网地址。然而，在实践中，它们的子网地址经常有许多共同之处。为了理解其中的道理，我们来关注在全球因特网中是如何处理编址的。

因特网的地址分配策略被称为无类别域间路由选择（Classless Interdomain Routing, CIDR）[RFC 4632]。CIDR 将子网寻址的概念一般化了。因为对于子网寻址，32 比特的 IP 地址被划分为两部分，并且也具有点分十进制数形式 $a.b.c.d/x$ ，其中 x 指示了地址的第一部分中的比特数。

形式为 $a.b.c.d/x$ 的地址的 x 最高比特构成了 IP 地址的网络部分，并且经常被称为该地址的**前缀**（prefix）（或网络前缀）。一个组织通常被分配一块连续的地址，即具有相同前缀的一段地址（参见“实践原则”）。在这种情况下，该组织内部的设备的 IP 地址将共享共同的前缀。当我们在 4.6 节中论及因特网的 BGP 路由选择协议时，将看到该组织网络外部的路由器仅考虑前面的前缀比特 x 。这相当大地减少了在这些路由器中转发表的长度，因为形式为 $a.b.c.d/x$ 单一表项足以将数据报转发到该组织内的任何目的地。

实践原则

这是一个 ISP 将 8 个组织连接到因特网的例子，它也很好说明了仔细分配 CIDR 化的地址有利于路由选择的道理。如图 4-18 所示，假设该 ISP（我们称之为 Fly-By-Night-ISP）向外界通告，它应该发送所有地址的前 20 比特与 200.23.16.0/20 相符的数据报。外界的其他部分不需要知道在地址块 200.23.16.0/20 内实际上还存在 8 个其他组织，每个组织有自己的子网。这种使用单个网络前缀通告多个网络的能力通常称为**地址聚合**（address aggregation），也称为**路由聚合**（route aggregation）或**路由摘要**（route summarization）。

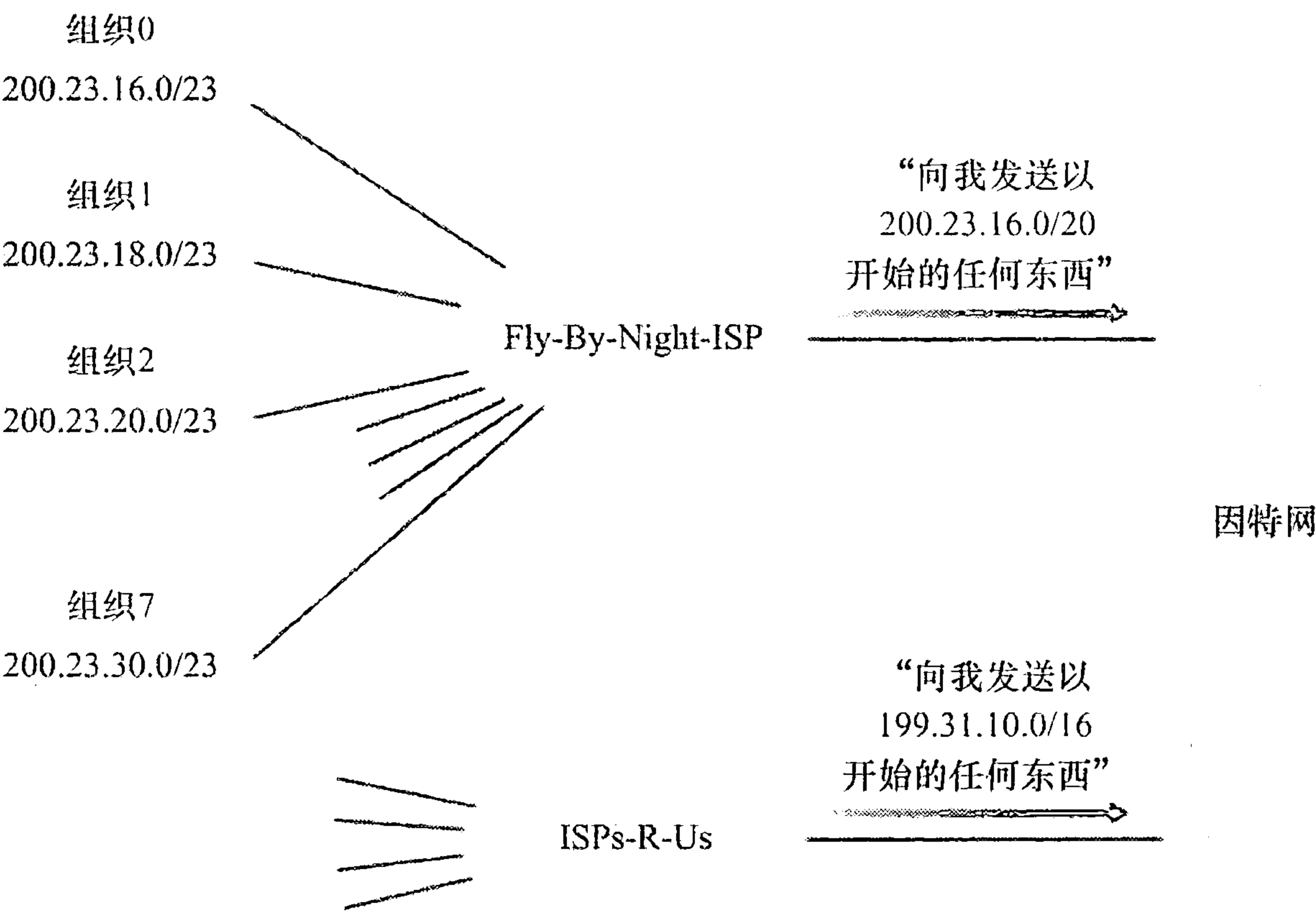


图 4-18 层次编址与路由聚合

当地址按块分给 ISP，然后又由 ISP 分给客户组织时，地址聚合工作极为有效。但是当地址不是按这样的层次方式分配时，会出现什么情况呢？例如，如果 Fly-By-Night-ISP 获取了 ISPs-R-Us，然后让组织 1 通过它辅助的 ISPs-R-Us 与因特网相连，将会发生什么情况呢？如图 4-18 中所示，该辅助的 ISPs-R-Us 拥有地址块 199.31.0.0/16，但很遗憾的是组织 1 的 IP 地址在该地址块之外。这里可以采取什么措施呢？组织 1 无疑可以将其所有的路由器和主机重新编号，使得地址在 ISPs-R-Us 的地址块内。但这是一种代价很高的方案，而且组织 1 将来也许还会从 ISPs-R-Us 更换到另一个 ISP。通常采用的

做法是，组织 1 保持其 IP 地址在 200.23.18.0/23 内。在这种情况下，如图 4-19 所示，Fly-By-Night-ISP 继续通告地址块 200.23.16.0/20，并且 ISPs-R-Us 也继续通告地址块 199.31.0.0/16。然而，ISPs-R-Us 现在还要通告组织 1 的地址块 200.23.18.0/23。当在更大的因特网上的其他路由器看见地址块 200.23.16.0/20（来自 Fly-By-Night-ISP）和 200.23.18.0/23（来自 ISPs-R-Us），并且想路由选择到在地址块 200.23.18.0/23 内的一个地址时，它们将使用一个最长前缀匹配（参见 4.2.2 节），并朝着 ISPs-R-Us 路由，因为它通告了与目的地址相匹配的最长（最具体）的地址前缀。

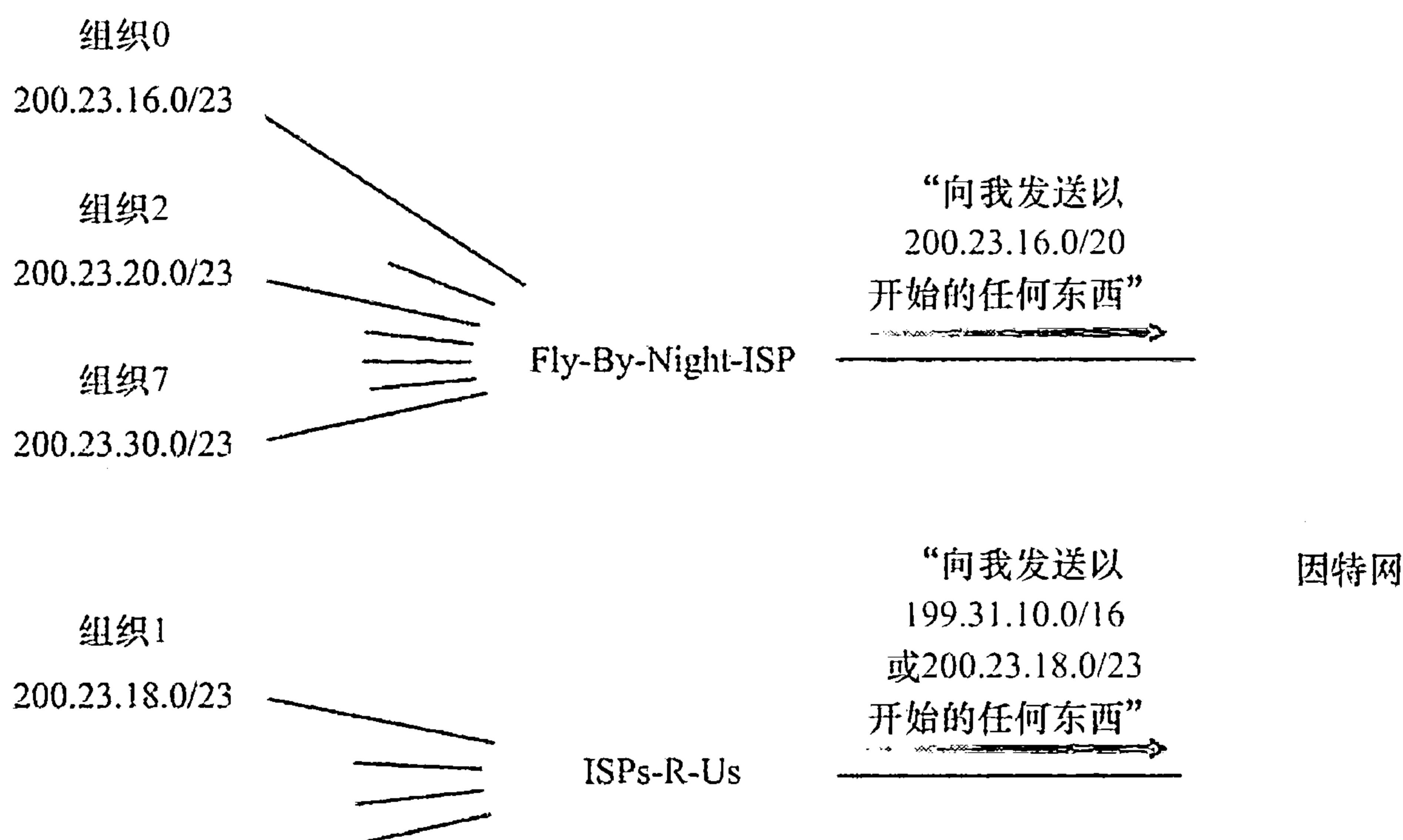


图 4-19 ISPs-R-Us 具有到组织 1 的一条更具体的路由

一个地址的剩余 $32 - x$ 比特可认为是用于区分该组织内部设备的，其中的所有设备具有相同的网络前缀。当该组织内部的路由器转发分组时，才会考虑这些比特。这些较低阶比特可能（或可能不）具有另外的子网结构，如前面所讨论的那样。例如，假设某 CIDR 化的地址 $a.b.c.d/21$ 的前 21 比特定义了该组织的网络前缀，它对该组织中的所有主机的 IP 地址来说是共同的。其余的 11 比特标识了该组织内的主机。该组织的内部结构可以采用这样的方式，使用这最右边的 11 比特在该组织中划分子网，就像前面所讨论的那样。例如， $a.b.c.d/24$ 可能表示该组织内的特定子网。

在 CIDR 被采用之前，IP 地址的网络部分被限制为长度为 8、16 或 24 比特，这是一种称为分类编址（classful addressing）的编址方案，这是因为具有 8、16 和 24 比特子网地址的子网分别被称为 A、B 和 C 类网络。一个 IP 地址的网络部分正好为 1、2 或 3 字节的要求，已经在支持数量迅速增加的具有小规模或中等规模子网的组织方面出现了问题。一个 C 类（/24）子网仅能容纳多达 $2^8 - 2 = 254$ 台主机（ $2^8 = 256$ ，其中的两个地址预留用于特殊用途），这对于许多组织来说太小了。然而一个 B 类（/16）子网可支持多达 65 534 台主机，又太大了。在分类编址方法下，比方说一个有 2000 台主机的组织通常被分给一个 B 类（/16）地址。这就导致了 B 类地址空间的迅速损耗以及所分配的地址空间的利用

率低。例如，为具有 2000 台主机的组织分配了一个 B 类地址，就具有足以支持多达 65 534 个接口的地址空间，剩下的超过 63 000 个地址却不能被其他组织使用。

如果还不提及另一种类型的 IP 地址，即 IP 广播地址 255.255.255.255，那将是我们的不负责任。当一台主机发出一个目的地址为 255.255.255.255 的数据报时，该报文会交付给同一个网络中的所有主机。路由器也会有选择地向邻近的子网转发该报文（虽然它们通常不这样做）。

此时已经详细地学习 IP 编址，我们需要知道主机或子网最初是如何得到它们的地址的。我们先看一个组织是如何为其设备得到一个地址块的，然后再看一个设备（如一台主机）是如何从某组织的地址块中分配到一个地址的。

1. 获取一块地址

为了获取一块 IP 地址用于一个组织的子网，某网络管理员也许首先会与他的 ISP 联系，该 ISP 可能会从已分给它的更大地址块中提供一些地址。例如，该 ISP 也许自己已被分配了地址块 200.23.16.0/20。该 ISP 可以依次将该地址块分成 8 个长度相等的连续地址块，为本 ISP 支持的最多达 8 个组织中的一个分配这些地址块中的一块，如下所示。（为了便于查看，我们已将这些地址的网络部分加了下划线。）

ISP 的地址块	200.23.16.0/20	<u>11001000 00010111 00010000</u> 00000000
组织 0	200.23.16.0/23	<u>11001000 00010111 00010000</u> 00000000
组织 1	200.23.18.0/23	<u>11001000 00010111 00010010</u> 00000000
组织 2	200.23.20.0/23	<u>11001000 00010111 00010100</u> 00000000
.....		
组织 7	200.23.30.0/23	<u>11001000 00010111 00011110</u> 00000000

尽管从一个 ISP 获取一组地址是得到一块地址的一种方法，但这不是唯一的方法。显然，必须还有一种方法供 ISP 本身得到一块地址。是否有一个全球性的权威机构，它具有管理 IP 地址空间并向各 ISP 和其他组织分配地址块的最终责任呢？的确有一个！IP 地址由因特网名字和编号分配机构（Internet Corporation for Assigned Names and Numbers, ICANN）[ICANN 2012] 管理，管理规则基于 [RFC 2050]。非营利的 ICANN 组织 [NTIA 1998] 的作用不仅是分配 IP 地址，还管理 DNS 根服务器。它还有一项容易引起争论的工作，即分配域名与解决域名纷争。ICANN 向区域性因特网注册机构（如 ARIN、RIPE、APNIC 和 LACNIC）分配地址，这些机构一起形成了 ICANN 的地址支持组织 [ASO-ICANN 2012]，处理本地域内的地址分配/管理。

2. 获取主机地址：动态主机配置协议

某组织一旦获得了一块地址，它就可为本组织内的主机与路由器接口逐个分配 IP 地址。系统管理员通常手工配置路由器中的 IP 地址（常常在远程通过网络管理工具进行配置）。主机地址也能手动配置，但是这项任务目前通常更多的是使用动态主机配置协议（Dynamic Host Configuration, DHCP）[RFC 2131] 来完成。DHCP 允许主机自动获取（被分配）一个 IP 地址。网络管理员能够配置 DHCP，以使某给定主机每次与网络连接时能得到一个相同的 IP 地址，或者某主机将被分配一个临时的 IP 地址（temporary IP address），该地址在每次与网络连接时也许是不同的。除了主机 IP 地址分配外，DHCP 还允许一台主机得知其他信息，例如它的子网掩码、它的第一跳路由器地址（常称为默认网关）与它的

本地 DNS 服务器的地址。

由于 DHCP 具有能将主机连接进一个网络的网络相关方面的自动能力，故它又常被称为即插即用协议（plug-and-play protocol）。这种能力对于网络管理员来说非常有吸引力，否则他将不得不手工执行这些任务！DHCP 还广泛地使用于住宅因特网接入网与无线局域网中，其中的主机频繁地加入和离开网络。例如，考虑一个学生带着便携机从宿舍到图书馆再到教室。很有可能在每个位置，这个学生将连接到一个新的子网，因此在每个位置都需要一个新的 IP 地址。DHCP 在此情形下是最理想的方法，因为有许多用户来来往往，仅在有限的时间内需要地址。类似地，DHCP 在住宅 ISP 接入网中是有用的。举一个例子，一个住宅区 ISP 有 2000 个客户，但不会有超过 400 个客户同时在线。在这种情况下，动态地分配地址的 DHCP 服务器不需要一个含 2048 个地址的块，而仅需一个含 512 个地址的块（例如像形式为 $a.b.c.d/23$ 的块）。当主机加入或离开时，DHCP 服务器要更新其可用的 IP 地址表。每当一台主机加入时，DHCP 服务器从其当前可用的地址池中分配一个任意的地址给它；每当一台主机离开时，其地址便被收回这个池中。

DHCP 是一个客户 - 服务器协议。客户通常是新到达的主机，它要获得包括自身使用的 IP 地址在内的网络配置信息。在最简单场合下，每个子网（在图 4-17 的编址意义下）将具有一台 DHCP 服务器。如果在某子网中没有服务器，则需要一个 DHCP 中继代理（通常是一台路由器），这个代理知道用于该网络的 DHCP 服务器的地址。图 4-20 显示了连接到子网 223.1.2/24 的一台 DHCP 服务器，具有一台提供中继代理服务的路由器，它为连接到子网 223.1.1/24 和 223.1.3/24 的到达客户提供 DHCP 服务。在我们下面的讨论中，将假定 DHCP 服务器在该子网上是可供使用的。

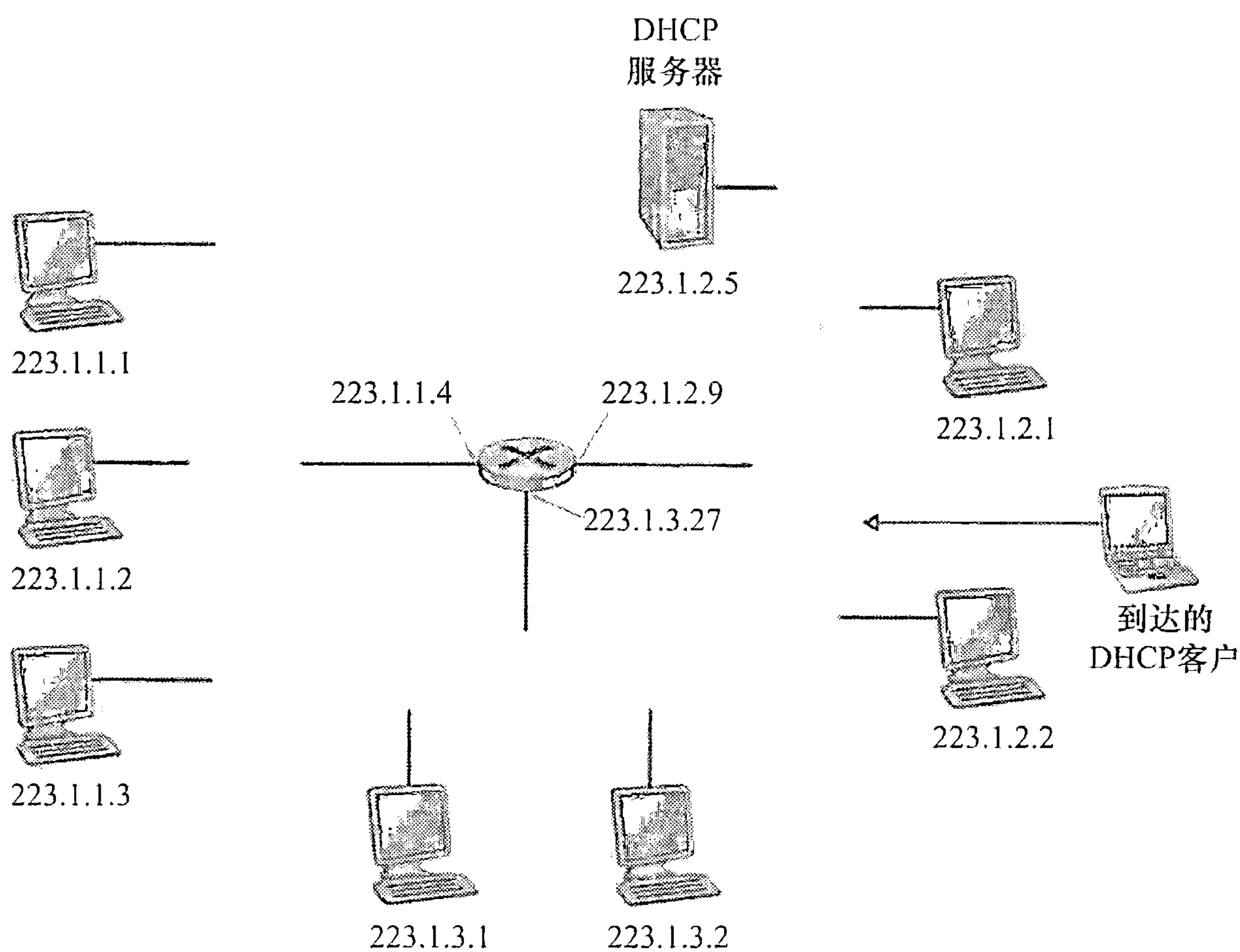


图 4-20 DHCP 客户 - 服务器场景

对于一台新到达的主机而言，针对图 4-20 所示的网络设置，DHCP 协议是一个 4 个步骤的过程，如图 4-21 中所示。在这幅图中，yiaddr（表示“你的因特网地址”之意）指示被分配给该新到达客户的地址。

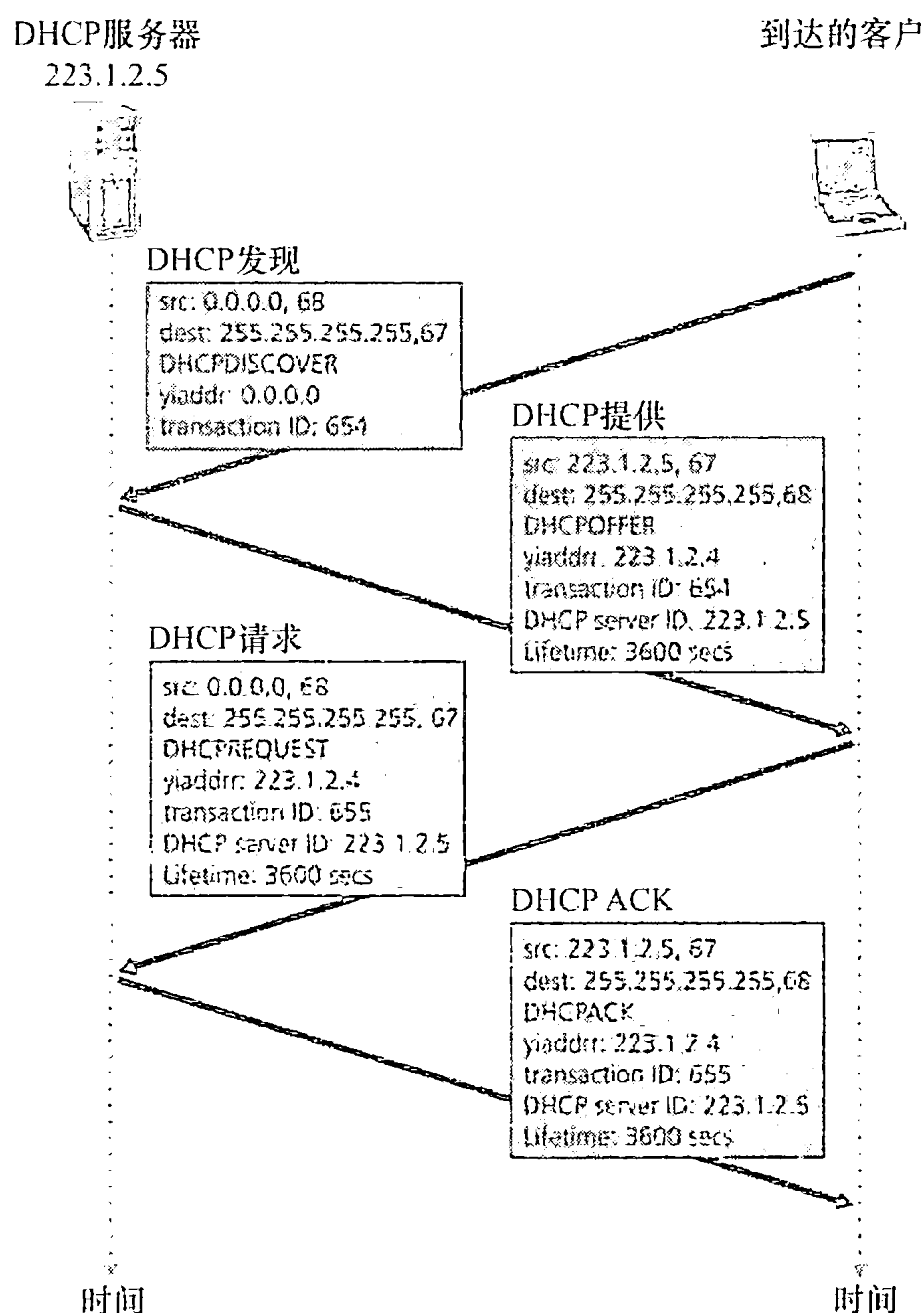


图 4-21 DHCP 客户 - 服务器交互

这 4 个步骤是：

- DHCP 服务器发现。一台新到的主机的首要任务是发现一个要与其交互的 DHCP 服务器。这可通过使用一个 DHCP 发现报文（DHCP discover message）来完成，客户在 UDP 分组中向端口 67 发送该发现报文。但是这个数据报应发给谁呢？主机甚至不知道它所连接网络的 IP 地址，更不用说用于该网络的 DHCP 服务器地址了。在这种情况下，DHCP 客户生成包含 DHCP 发现报文的 IP 数据报，其中使用广播目的地址 255.255.255.255 并且使用“本主机”源地址 0.0.0.0。DHCP 客户将该 IP 数据报传递给链路层，链路层然后将该帧广播到所有与该子网连接的子网（我们将在 5.4 节中涉及链路层广播的细节）。
- DHCP 服务器提供。DHCP 服务器收到一个 DHCP 发现报文时，用一个 DHCP 提供报文（DHCP offer message）向客户作出响应，仍然使用 IP 广播地址 255.255.255.255。（你也许要思考一下这个服务器为何也必须采用广播。）因为在子网中可能有几个 DHCP 服务器，该客户也许会发现它处于能在几个提供者之间进行选择的优越位置。每台服务器提供的报文包含有收到的发现报文的事务 ID、向客户推荐的 IP 地址、网络掩码以及 IP 地址租用期（address lease time），即 IP 地址有效的时间量。服务器租用期通常设置为几小时或几天 [Droms 2002]。
- DHCP 请求。新到达的客户从一个或多个服务器提供中选择一个，并向选中的服务器提供用一个 DHCP 请求报文（DHCP request message）进行响应，回显配置参数。

- DHCP ACK。服务器用 DHCP ACK 报文 (DHCP ACK message) 对 DHCP 请求报文进行响应, 证实所要求的参数。

一旦客户收到 DHCP ACK 后, 交互便完成了, 并且该客户能够在租用期内使用 DHCP 分配的 IP 地址。因为客户可能在该租用期超时时还希望使用这个地址, 所以 DHCP 还提供了一种机制以允许客户更新它对一个 IP 地址的租用。

考虑到作为人工配置主机的 IP 地址的替代方法, DHCP 的即插即用能力的价值是显而易见的。考虑这样的情况: 一个学生带着便携机从教室移动到图书馆、宿舍, 每到新的位置就加入一个新的网络并获得一个新的 IP 地址。如果在每个位置都要系统管理员重新配置便携机, 而且很少有学生 (上过计算机网络课程的学生除外) 有手工配置其计算机的经验, 这种情况简直是不可想象的。然而, 从移动性角度看, DHCP 确实有不足之处。因为每当结点连到一个新子网时, 要从 DHCP 得到一个新的 IP 地址, 当一个移动结点在子网之间移动时, 就不能维持与远程应用之间的 TCP 连接。在第 6 章中, 我们将研究移动 IP, 它是一种对 IP 基础设施的近期扩展, 允许移动结点在网络之间移动时能使用其单一永久的地址。有关 DHCP 的其他细节可在 [Droms 2002] 与 [dhc 2012] 中找到。一个 DHCP 开放源码参考实现可从因特网系统协会 [ISC 2012] 得到。

3. 网络地址转换

讨论了有关因特网地址和 IPv4 数据报格式后, 我们现在可清楚地认识到每个 IP 使能的设备都需要一个 IP 地址。随着所谓小型办公室、家庭办公室 (Small Office, Home Office, SOHO) 子网的大量出现, 这看起来意味着每当一个 SOHO 想安装一个 LAN 以互联多台机器时, 需要 ISP 分配一组地址以供该 SOHO 的所有机器使用。如果该子网变大了 (例如家里的孩子们不仅有自己的计算机, 而且还有智能手机、联网游戏机), 则需要分配一块较大的地址。但如果 ISP 已经为 SOHO 网络当前地址范围分配过一块连续地址该怎么办呢? 并且家庭主人要 (或应该需要) 首先知道管理 IP 地址的典型方法有哪些呢? 幸运的是, 有一种简单的方法越来越广泛地在这些情况场合: 网络地址转换 (Network Address Translation, NAT) [RFC 2663; RFC 3022; Zhang 2007]。

图 4-22 显示了一台 NAT 使能路由器的运行情况。位于家中的 NAT 使能的路由器有一个接口, 该接口是图 4-22 中右侧所示家庭网络的一部分。在家庭网络内的编址就像我们在上面看到的完全一样, 其中的所有 4 个接口都具有相同的网络地址 10.0.0.0/24。地址空间 10.0.0.0/8 是在 [RFC 1918] 中保留的 3 部分 IP 地址空间之一, 这些地址用于如图 4-22 中的家庭网络等专用网络或具有专用地址的地域 (realm)。具有专用地址的地域是指其地址仅对该网络中的设备有意义的网络。为了明白它为什么重要, 考虑有数十万家庭网络这样的事实, 许多使用了相同的地址空间 10.0.0.0/24。在一个给定家庭网络中的设备能够使用 10.0.0.0/24 编址彼此发送分组。然而, 转发到家庭网络之外进入更大的全球因特网的分组显然不能使用这些地址 (或作为源地址, 或作为目的地址), 因为有数十万的网络使用着这块地址。这就是说, 10.0.0.0/24 地址仅在给定的网络中才有意义。但是如果专用地址仅在给定的网络中才有意义的话, 当向或从全球因特网发送或接收分组, 地址在何处才必须是唯一的呢? 答案在于理解 NAT。

NAT 使能路由器对于外部世界来说甚至不像一台路由器。NAT 路由器对外界的行为反过来就如同一个具有单一 IP 地址的单一设备。在图 4-22 中, 所有离开家庭路由器流向

更大因特网的报文都拥有一个源 IP 地址 138.76.29.7，且所有进入家庭的报文都拥有同一个目的 IP 地址 138.76.29.7。从本质上讲，NAT 使能路由器对外界隐藏了家庭网络的细节。（另外，你也许想知道家庭网络计算机是从哪儿得到其地址，路由器又是从哪儿得到它的单一 IP 地址的。在通常的情况下，答案是相同的，即 DHCP！路由器从 ISP 的 DHCP 服务器得到它的地址，并且路由器运行一个 DHCP 服务器，为位于 NAT-DHCP 路由器控制的 家庭网络地址空间中的计算机提供地址。）

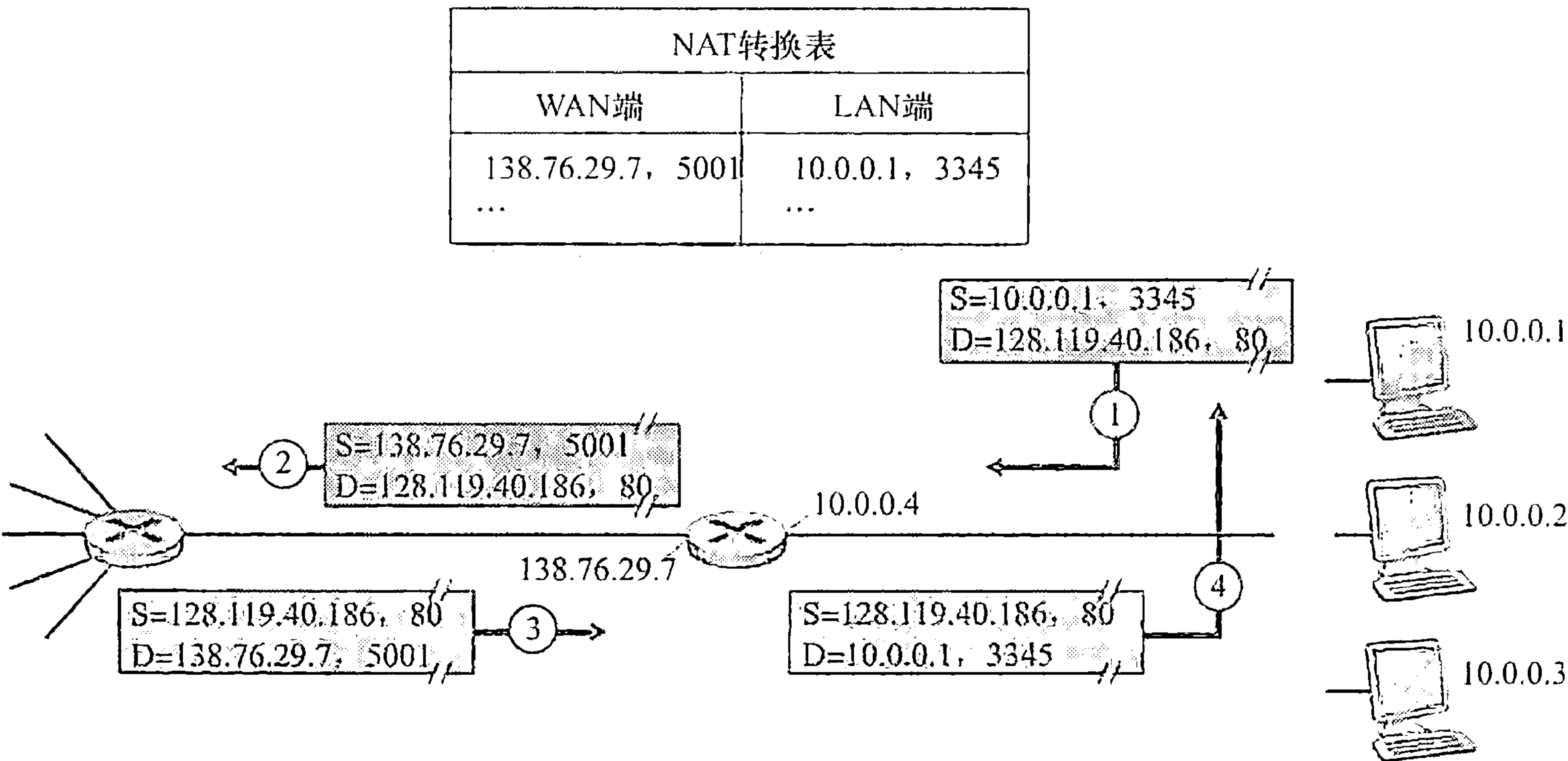


图 4-22 网络地址转换

如果从广域网到达 NAT 路由器的所有数据报都有相同的目的 IP 地址（特别是对 NAT 路由器广域网一侧的接口），那么该路由器怎样知道它应将某个分组转发给哪个内部主机呢？技巧就是使用在 NAT 路由器上的一张 NAT 转换表（NAT translation table），并且在表项中包含了端口号及其 IP 地址。

考虑图 4-22 中的例子。假设一个用户坐在家庭网络主机 10.0.0.1 旁，请求 IP 地址为 128.119.40.186 的某台 Web 服务器（端口 80）上的一个 Web 页面。主机 10.0.0.1 为其指派了（任意）源端口号 3345 并将该数据报发送到 LAN 中。NAT 路由器收到该数据报，为该数据报生成一个新的源端口号 5001，将源 IP 替代为其广域网一侧接口的 IP 地址 138.76.29.7，且将源端口 3345 更换为新端口 5001。当生成一个新的源端口号时，NAT 路由器可选择任意一个当前未在 NAT 转换表中的源端口号。（注意到因为端口号字段为 16 比特长，NAT 协议可支持超过 60 000 个并行使用路由器广域网一侧 IP 地址的连接！）路由器中的 NAT 也在它的 NAT 转换表中增加一表项。Web 服务器并不知道刚到达的包含 HTTP 请求的数据报已被 NAT 路由器进行了改装，它会发回一个响应报文，其目的地址是 NAT 路由器的 IP 地址，其目的端口是 5001。当该报文到达 NAT 路由器时，路由器使用目的 IP 地址与目的端口号从 NAT 转换表中检索出家庭网络浏览器使用的适当 IP 地址（10.0.0.1）和目的端口号（3345）。于是，路由器改写该数据报的目的 IP 地址与目的端口号，并向家庭网络转发该数据报。

NAT 在近几年已得到了广泛的应用。但是，我们应当提及的是，许多 IETF 团体中的纯化论者大声疾呼反对 NAT。第一，他们认为端口号是用于进程编址的，而不是用于主机编址的。（这种违规用法对于运行在家庭网络中的服务器来说确实会引起问题，因为我们

已在第2章看到，服务器进程在周知端口号上等待入请求。)第二，他们认为路由器通常仅应当处理高达第三层的分组。第三，他们认为 NAT 协议违反了所谓端到端原则，即主机彼此应相互直接对话，结点不应介入修改 IP 地址与端口号。第四，他们认为应使用 IPv6（参见 4.4.4 节）来解决 IP 地址短缺问题，而不是不计后果地用一种如 NAT 之类的权宜之计来修补存在的问题。但不管喜欢与否，NAT 已成为因特网的一个重要组件。

NAT 的另一个重要问题是它妨碍 P2P 应用程序，包括 P2P 文件共享应用和 P2PIP 语音应用。第2章讲过在一个 P2P 应用程序中，任何参与对等方 A 应当能够对任何其他参与对等方 B 发起一条 TCP 连接。该问题的实质在于如果对等方 B 在一个 NAT 后面，它不能充当服务器并接收 TCP 连接。如我们将在课后习题中所见，如果对等方 A 不在一个 NAT 的后面，则该 NAT 问题能够绕过去。在这种情况下，对等方 A 能够首先通过一个中间对等方 C 与对等方 B 联系，其中 C 不位于 NAT 之后并与 B 已经创建了一条进行中的 TCP 连接。对等方 A 则能够经对等方 C 请求对等方 B，发起直接返回对等方 A 的一条 TCP 连接。一旦对等方 A 和 B 之间创建一条直接的 P2P TCP 连接，这两个对等方就能够交换报文或文件。这种雇佣关系被称为**连接反转**（connection reversal），实际上被许多 P2P 应用程序用于 NAT 穿越（NAT traversal）。如果对等方 A 和对等方 B 都在它们自己的 NAT 后面，这种情况有些棘手，但是能够使用应用程序进行中继处理，正如我们在第2章中对于 Skype 中继所见到的那样。

4. UPnP

NAT 穿越正越来越多地由通用即插即用（UPnP）提供，UPnP 是一种允许主机发现并配置邻近 NAT 的协议 [UPnP Forum 2012]。UPnP 要求主机和 NAT 都是 UPnP 兼容的。使用 UPnP，在主机上运行的应用程序能够为某些请求的公共端口号请求一个 NAT 映射，该映射位于其（专用 IP 地址，专用端口号）和（公共 IP 地址，公共端口号）之间。如果某 NAT 接受该请求并生成映射，则来自外部的结点能够发起到（公共 IP 地址，公共端口号）的 TCP 连接。此外，UPnP 让该应用程序知道（公共 IP 地址，公共端口号），因此该应用程序能够向外部世界通告它。

举一个例子，假定你的主机位于一个 UPnP 使能的 NAT 背后，具有专用地址 10.0.0.1 并且在端口 3345 上运行 BitTorrent。另外假定该 NAT 的公共 IP 地址是 138.76.29.7。你的 BitTorrent 应用程序自然要能够接受来自其他主机的连接，使得它能够同其他主机对换块。此后，你主机上的 BitTorrent 应用程序请求 NAT 产生一个“洞”，将（10.0.0.1，3345）映射到（138.76.29.7，5001）。（该应用程序选择了公共端口号 5001。）在你主机中的 BitTorrent 应用程序也能够向它的追踪器通告它在（138.76.29.7，5001）可供使用。以这种方式，运行 BitTorrent 的一台外部主机能够联系该追踪器，并知道你的 BitTorrent 应用程序正运行在（138.76.29.7，5001）。该外部主机能够向（138.76.29.7，5001）发送 TCP SYN 分组。当 NAT 接收到该 SYN 分组，它将改变分组中的目的 IP 地址和端口号，并通过 NAT 转发分组。

总而言之，UPnP 允许外部主机使用 TCP 或 UDP 向 NAT 化的主机发起通信会话。长期以来 NAT 一直对 P2P 应用程序十分不利；UPnP 由于提供了有效和健壮的 NAT 穿越解决方案，可能成为了 P2P 应用程序的救世主。这里我们对 NAT 和 UPnP 的讨论十分简要，对于 NAT 更为详细的讨论参见 [Huston 2004; Cisco NAT 2012]。

4.4.3 因特网控制报文协议

前面讲过因特网的网络层具有 3 个主要组件：在前面一节中讨论的 IP 协议；在 4.6 节中讨论的因特网路由选择协议（包括 RIP、OSPF 和 BGP）；以及成为本节主题的因特网控制报文协议（ICMP）。

ICMP 由 [RFC 792] 定义，被主机和路由器用来彼此沟通网络层的信息。ICMP 最典型的用途是差错报告。例如，当运行一个 Telnet、FTP 或 HTTP 会话时，你也许会遇到一些诸如“目的网络不可达”之类的错误报文。这种报文就是在 ICMP 中产生的。在某个位置，IP 路由器不能找到一条路径，以通往 Telnet、FTP 或 HTTP 应用所指定的主机。该路由器就会向你的主机创建和发出一个类型 3 的 ICMP 报文以指示该错误。

ICMP 通常被认为是 IP 的一部分，但从体系结构上讲它是位于 IP 之上的，因为 ICMP 报文是承载在 IP 分组中的。这就是说，ICMP 报文是作为 IP 有效载荷承载的，就像 TCP 与 UDP 报文段作为 IP 有效载荷被承载那样。类似地，当一台主机收到一个指明上层协议为 ICMP 的 IP 数据报时，它分解出该数据报的内容给 ICMP，就像分解出一个数据报的内容给 TCP 或 UDP 一样。

ICMP 报文有一个类型字段和一个编码字段，并且包含引起该 ICMP 报文首次生成的 IP 数据报的首部和前 8 字节内容（以便发送方能确定引发该差错的数据报）。在图 4-23 中显示了所选的 ICMP 报文类型。注意到 ICMP 报文并不仅是用于通知差错情况。

ICMP 类型	编码	描述
0	0	回显回答（对 ping 的回答）
3	0	目的网络不可达
3	1	目的主机不可达
3	2	目的协议不可达
3	3	目的端口不可达
3	6	目的网络未知
3	7	目的主机未知
4	0	源抑制（拥塞控制）
8	0	回显请求
9	0	路由器通告
10	0	路由器发现
11	0	TTL过期
12	0	IP 首部损坏

图 4-23 ICMP 报文类型

众所周知的 ping 程序发送一个 ICMP 类型 8 编码 0 的报文到指定主机。看到该回显（echo）请求，目的主机发回一个类型 0 编码 0 的 ICMP 回显回答。大多数 TCP/IP 实现直接在操作系统中支持 ping 服务器，即该服务器不是一个进程。[Stevens 1990] 的第 11 章提供了有关 ping 客户程序的源码。注意客户程序需要能够指示操作系统产生一个类型 8 编码 0 的 ICMP 报文。

另一个有趣的 ICMP 报文是源抑制报文。这种报文在实践中很少使用。其最初目的是执行拥塞控制，即使得拥塞的路由器向一台主机发送一个 ICMP 源抑制报文，以强制该主

机减小其发送速率。我们在第3章已看到，TCP有自己在运输层操作的拥塞控制机制，不需要利用网络层中的反馈信息（如ICMP源抑制报文）。

在第1章中我们介绍了Traceroute程序，该程序允许我们跟踪从一台主机到世界上任意一台其他主机之间的路由。有趣的是Traceroute是用ICMP报文来实现的。为了判断源和目的地之间所有路由器的名字和地址，源主机中的Traceroute向目的主机发送一系列普通的IP数据报。这些数据报的每个携带了具有一个不可达UDP端口号的UDP报文段。第一个数据报的TTL为1，第二个的TTL为2，第三个的TTL为3，依次类推。该源主机也为每个数据报启动定时器。当第 n 个数据报到达第 n 台路由器时，第 n 台路由器观察到这个数据报的TTL正好过期。根据IP协议规则，路由器丢弃该数据报并发送一个ICMP告警报文给源主机（类型11编码0）。该告警报文包含了路由器的名字与它的IP地址。当该ICMP报文返回源主机时，源主机从定时器得到往返时延，从ICMP报文中得到第 n 台路由器的名字与IP地址。

Traceroute源主机是怎样知道何时停止发送UDP报文段的呢？前面讲过源主机为它发送的每个报文段的TTL字段加1。因此，这些数据报之一将最终沿着这条路到达目的主机。因为该数据报包含了一个具有不可达端口号的UDP报文段，该目的主机将向源发送一个端口不可达的ICMP报文。当源主机收到这个特别的ICMP报文时，知道它不需要再发送另外的探测分组。（标准的Traceroute程序实际上用相同的TTL发送3个一组的分组；因此Traceroute输出对每个TTL提供了3个结果。）

以这种方式，源主机知道了位于它与目的主机之间的路由器数量和标识，以及两台主机之间的往返时延。注意Traceroute客户程序必须能够指令操作系统产生具有特定TTL值的UDP数据报，当ICMP报文到达时，也必须能够由它的操作系统进行通知。既然你已明白了Traceroute的工作原理，你也许想回去更多地使用它。

关注安全性

检查数据报：防火墙和入侵检测系统

假定你被赋予了管理家庭网络、部门网络、大学网络或公司网络的任务。知道你网络IP地址范围的攻击者，能够方便地在此范围中发送IP数据报进行寻址。这些数据报能够做各种不正当的事情，包括用ping搜索和端口扫描形成你的网络图，用恶意分组使易受攻击的主机崩溃，用纷至沓来的ICMP分组洪泛服务器，并且通过在分组中带有恶意软件感染主机。作为网络管理员，你准备做些什么来将这些能够在你的网络中发送恶意分组的坏家伙拒之门外呢？对抗恶意分组攻击的两种流行的防御措施是防火墙和入侵检测系统（IDS）。

作为一名网络管理员，你可能首先尝试在你的网络和因特网之间安装一台防火墙。（今天大多数接入路由器具有防火墙能力。）防火墙检查数据报和报文段首部字段，拒绝可疑的数据报进入内部网络。例如，一台防火墙可以被配置为阻挡所有的ICMP回显请求分组，从而防止了攻击者横跨你的IP地址范围进行传统的ping搜索。防火墙也能基于源和目的IP地址和端口号阻挡分组。此外，防火墙能够配置为跟踪TCP连接，仅许可属于批准连接的数据报进入。

IDS 能够提供另一种保护措施。IDS 通常位于网络的边界，执行“深度分组检查”，不仅检查数据报（包括应用层数据）中的首部字段，而且检查其有效载荷。IDS 具有一个分组特征数据库（这些特征是已知攻击的一部分）。随着新攻击的发现，该数据库自动更新特征。当分组通过 IDS 时，IDS 试图将分组的首部字段和有效载荷与其特征数据库中的特征相匹配。如果发现了这样的一种匹配，就产生一个告警。入侵防止系统（IPS）与 IDS 类似，只是除了产生告警外还实际阻挡分组。在第 8 章中，我们将更为详细地研究防火墙和 IDS。

防火墙和 IDS 能够全面保护你的网络免受所有攻击吗？答案显然是否定的，因为攻击者继续寻找特征还不能匹配的新攻击方法。但是防火墙和传统的基于特征的 IDS 在保护你的网络不受已知攻击入侵方面是有用的。

4.4.4 IPv6

在 20 世纪 90 年代早期，因特网工程任务组就开始致力于开发一种替代 IPv4 的协议。该努力的最初动机是因为以下现实：由于新的子网和 IP 结点以惊人的增长率连到因特网上（并被分配唯一的 IP 地址），32 比特的 IP 地址空间即将用尽。为了应对这种对大 IP 地址空间的需求，开发了一种新的 IP 协议，即 IPv6。IPv6 的设计者们还利用这次机会，在 IPv4 积累的运行经验基础上加进和强化了 IPv4 的其他方面。

IPv4 地址在什么时候会被完全分配完（因此没有新的网络再能与因特网相连）是一个相当有争议的问题。IETF 的地址寿命期望工作组的两位负责人分别估计地址将于 2008 年和 2018 年用完 [Solensky 1996]。在 2011 年 2 月，IANA 向一个区域注册机构分配完了未分配 IPv4 地址的最后剩余地址池。这些注册机构在它们的地址池中还有可用的 IPv4 地址，一旦用完这些地址，从中央池中将再也分配不出更多的可用地址块了 [Huston 2011a]。尽管在 20 世纪 90 年代中期对 IPv4 地址耗尽的估计表明，IPv4 地址空间耗尽的期限还有可观的时间，但人们认识到，如此大规模地部署一项新技术将需要可观的时间，因此开始了下一代 IP（Next Generation IP，IPng）的工作 [Bradner 1996；RFC 1752]。这种工作成果就是 IP 版本 6（IPv6）的规范 [RFC 2460]，这是我们下面将要讨论的主题。（一个经常问到的问题是：IPv5 出了什么情况？人们最初预想 ST-2 协议将成为 IPv5，但 ST-2 后来被舍弃了。）有关 IPv6 的优秀信息来源见 [Huitema 1998；IPv6 2012]。

1. IPv6 数据报格式

IPv6 数据报的格式如图 4-24 所示。

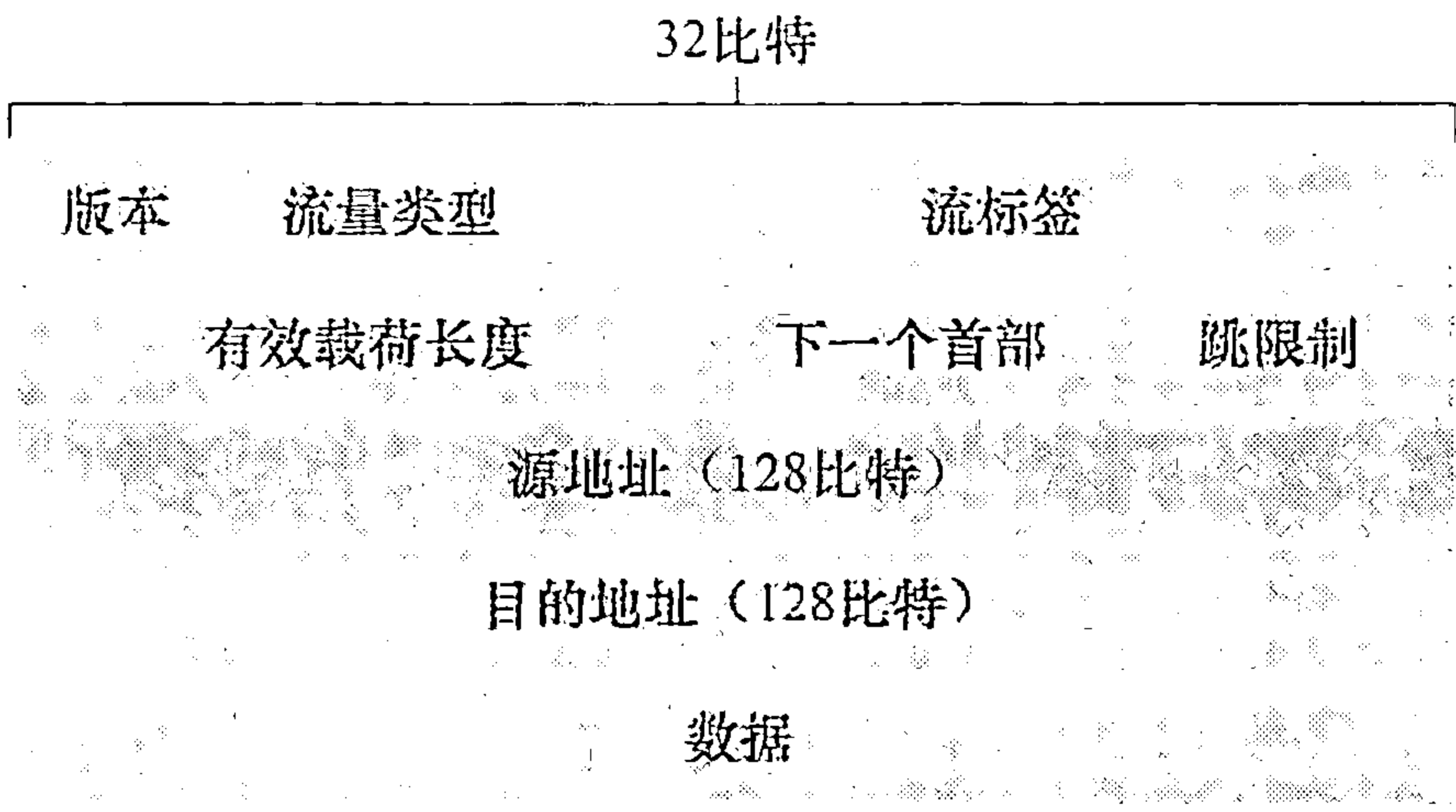


图 4-24 IPv6 数据报格式

IPv6 中引入的最重要的变化显示在其数据报格式中：

- 扩大的地址容量。IPv6 将 IP 地址长度从 32 比特增加到 128 比特。这就确保全世界将不会用尽 IP 地址。现在，地球上的每个沙砾都可以用 IP 地址寻址了。除了单播与多播地址以外，IPv6 还引入了一种称为任播地址（anycast address）的新型地址，这种地址可以使数据报交付给一组主机中的任意一个。（例如，这种特性可用于向一组包含给定文档的镜像站点中的最近的一个发送一个 HTTP GET 报文。）
- 简化高效的 40 字节首部。如下面讨论的那样，许多 IPv4 字段已被舍弃或作为选项。因而所形成的 40 字节定长首部允许更快地处理 IP 数据报。一种新的选项编码允许进行更灵活的选项处理。
- 流标签与优先级。IPv6 有一个难以捉摸的流（flow）定义。RFC 1752 与 RFC 2460 中描述道，该字段可用于“给属于特殊流的分组加上标签，这些特殊流是发送方要求进行特殊处理的流，如一种非默认服务质量或需要实时服务的流”。例如，音频与视频传输就可能被当作一个流。在另一方面，更为传统的应用（如文件传输和电子邮件，就不可能被当作流。由高优先级用户（如某些为使其流量得到更好服务而付费的用户）承载的流量也有可能被当作一个流。然而，IPv6 的设计者们显然已预见到最终需要能够区分这些流，即使流的确切含义还未完全确定。IPv6 首部中还有一个 8 比特的流量类型字段，该字段就像 IPv4 中的 TOS 字段，可用于给出一个流中某些数据报的优先级，以便指明某些应用的数据报（如 ICMP 分组）比其他应用的数据报（如网络新闻）有更高的优先权。

如上所述，比较图 4-24 与图 4-13 就可看出，IPv6 数据报的结构更简单、更高效。以下是在 IPv6 中定义的字段。

- 版本。该 4 比特字段用于标识 IP 版本号。毫不奇怪，IPv6 将该字段值设为 6。注意到将该字段值置为 4 并不能创建一个合法的 IPv4 数据报。（如果这样的话，事情就简单多了，参见下面有关从 IPv4 向 IPv6 迁移的讨论。）
- 流量类型。该 8 比特字段与我们在 IPv4 中看到的 TOS 字段的含义相似。
- 流标签。如上面讨论过的那样，该 20 比特的字段用于标识一条数据报的流。
- 有效载荷长度。该 16 比特值作为一个无符号整数，给出了 IPv6 数据报中跟在定长的 40 字节数据报首部后面的字节数量。
- 下一个首部。该字段标识数据报中的内容（数据字段）需要交付给哪个协议（如 TCP 或 UDP）。该字段使用与 IPv4 首部中协议字段相同的值。
- 跳限制。转发数据报的每台路由器将对该字段的内容减 1。如果跳限制计数到达 0 时，则该数据报将被丢弃。
- 源地址和目的地址。IPv6 128 比特地址的各种格式在 RFC 4291 中进行了描述。
- 数据。这是 IPv6 数据报的有效载荷部分。当数据报到达目的地时，该有效载荷就从 IP 数据报中移出，并交给在下一个首部字段中指定的协议处理。

以上讨论说明了 IPv6 数据报中包括的各字段的用途。图 4-24 中的 IPv6 数据报格式与图 4-13 中的 IPv4 数据报格式进行比较，我们就会注意到，在 IPv4 数据报中出现的几个字段在 IPv6 数据报中已不复存在：

- 分片/重新组装。IPv6 不允许在中间路由器上进行分片与重新组装。这种操作只能在源与目的地上执行。如果路由器收到的 IPv6 数据报因太大而不能转发到出链路

上的话，则路由器只需丢掉该数据报，并向发送方发回一个“分组太大”的 ICMP 差错报文即可（见下文）。于是发送方能够使用较小长度的 IP 数据报重发数据。分片与重新组装是一个耗时的操作，将该功能从路由器中删除并放到端系统中，大大加快了网络中的 IP 转发速度。

- 首部检验和。因为因特网层中的运输层（如 TCP 与 UDP）和数据链路层（如以太网）协议执行了检验操作，IP 设计者大概觉得在网络层中具有该项功能实属多余，可以将其去除。再次强调的是，快速处理 IP 分组是关注的重点。在 4.4.1 节中我们讨论 IPv4 时讲过，由于 IPv4 首部中包含有一个 TTL 字段（类似于 IPv6 中的跳限制字段），所以在每台路由器上都需要重新计算 IPv4 首部检验和。就像分片与重新组装一样，在 IPv4 中这也是一项耗时的操作。
- 选项。选项字段不再是标准 IP 首部的一部分了。但它并没有消失，而是可能出现在 IPv6 首部中由“下一个首部”指出的位置上。这就是说，就像 TCP 或 UDP 协议首部能够是 IP 分组中的“下一个首部”，选项字段也能是“下一个首部”。删除选项字段使得 IP 首部成为定长的 40 字节。

我们在 4.4.3 节讨论时讲过，IP 结点使用 ICMP 协议来报告差错情况，并向端系统提供有限的信息（如对一个 ping 报文的回显回答）。RFC 4443 中定义了一种用于 IPv6 的新版 ICMP。除了能识别现存的 ICMP 类型和编码定义外，由于 IPv6 新增功能的需要，ICMPv6 还增加了新的类型和编码。其中包括“分组太大”类型与“未识别的 IPv6 选项”错误编码。另外，ICMPv6 还包含了我们将在 4.7 节中学习的因特网组管理协议（IGMP）。IGMP 用于管理主机加入和离开多播组，它在 IPv4 中曾是一个与 ICMP 分开的独立协议。

2. 从 IPv4 到 IPv6 的迁移

既然我们已了解了 IPv6 的技术细节，那么我们考虑一个非常实际的问题：基于 IPv4 的公共因特网如何迁移到 IPv6 呢？问题是，虽然新型 IPv6 使能系统可做成向后兼容，即能发送、路由和接收 IPv4 数据报，但已部署的 IPv4 使能系统却不能够处理 IPv6 数据报。可能采用以下几种方法。

一种可选的方法是宣布一个标志日，即指定某个日期和时间，届时因特网所有机器都关机并从 IPv4 升级到 IPv6。上次重大的技术迁移（为得到可靠的运输服务，从使用 NCP 迁移到使用 TCP）出现在差不多 25 年以前。即使回到那时 [RFC 801]——因特网很小且仍然由少数“奇才”管理着，人们也会认识到这样一个标志日是不可行的。一个涉及上亿台机器和上百万个网络管理员与用户的标志日现在更是不可想象的。RFC 4213 描述了两种方法（可单独使用，也可一起使用），可用于逐渐将 IPv6 主机和路由器整合进 IPv4 世界中（当然其长远目标是将所有 IPv4 结点最终向 IPv6 迁移）。

引入 IPv6 使能结点的最直接方式可能是一种双栈（dual-stack）方法，即使用该方法的 IPv6 结点还具有完整的 IPv4 实现。这样的结点在 RFC 4213 中被称为 IPv6/IPv4 结点，它有发送和接收 IPv4 与 IPv6 两种数据报的能力。当与 IPv4 结点互操作时，IPv6/IPv4 结点可使用 IPv4 数据报；当与 IPv6 结点互操作时，它又能使用 IPv6。IPv6/IPv4 结点必须具有 IPv6 与 IPv4 两种地址。此外，它们还必须能确定另一个结点是否是 IPv6 使能的或仅 IPv4 使能的。这个问题可使用 DNS（参见第 2 章）来解决，若要解析的结点名字是 IPv6 使能的，则 DNS 会返回一个 IPv6 地址，否则返回一个 IPv4 地址。当然，如果发出 DNS 请求的

结点是仅 IPv4 使能的，则 DNS 只返回一个 IPv4 地址。

在双栈方法中，如果发送方或接收方中任意一个仅为 IPv4 使能的，则必须使用 IPv4 数据报。因此，本质上两个 IPv6 使能的结点不应相互发送 IPv4 数据报。图 4-25 中图示了这种情况。假定结点 A 是 IPv6 使能的，且要向结点 F 发一个 IP 数据报，F 也是 IPv6 使能的。结点 A 和 B 能够交换 IPv6 数据报。然而，结点 B 必须生成一个 IPv4 数据报以便发给 C。当然，IPv6 数据报的数据字段可被复制到 IPv4 数据报的数据字段中，并且要做适当的地址映射。然而，在执行 IPv6 到 IPv4 的转换时，IPv6 数据报中一些 IPv6 特定的字段（如流标签字段）在 IPv4 数据报中无对应部分，这些字段的信息将会丢失。因此，即使 E 和 F 能交换 IPv6 数据报，从 D 到达 E 的 IPv4 数据报并不含有从 A 发出的初始 IPv6 数据报中的所有字段。

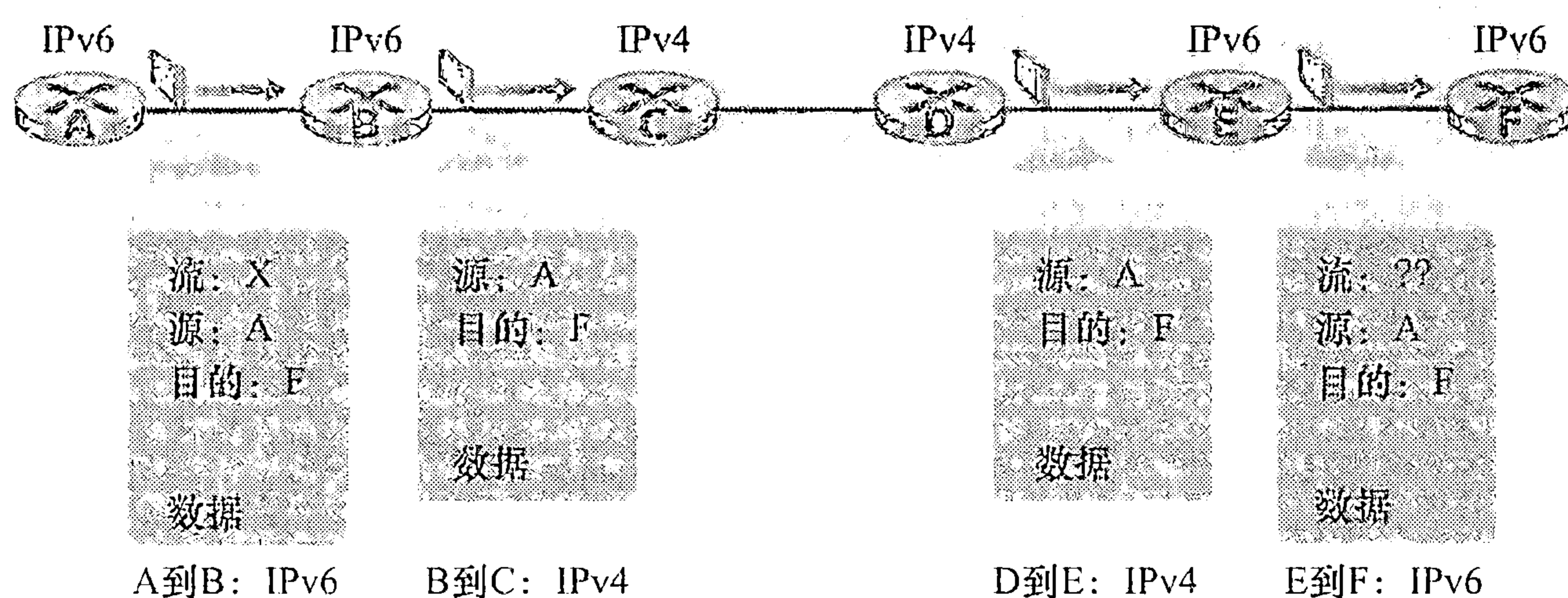


图 4-25 一种双栈方法

RFC 4213 中还讨论了另一种双栈方法，叫做建隧道（tunneling）。该方法能解决上述问题，允许如 E 接收源于 A 的 IPv6 数据报。隧道依据的基本思想如下：假定两个 IPv6 结点（如图 4-25 中的 B 和 E）要使用 IPv6 数据报进行交互，但它们是经由中间 IPv4 路由器互联的。我们将两台 IPv6 路由器之间的中间 IPv4 路由器的集合称为一个隧道（tunnel），如图 4-26 所示。借助于隧道，在隧道发送端的 IPv6 结点（如 B）可将整个 IPv6 数据报放到一个 IPv4 数据报的数据（有效载荷）字段中。于是，该 IPv4 数据报的地址设为指向隧道接收端的 IPv6 结点（如 E），再发送给隧道中的第一个结点（如 C）。隧道中的中间 IPv4 路由器在它们之间为该数据报提供路由，就像对待其他数据报一样，完全不知道该 IPv4 数据报自身就含有一个完整的 IPv6 数据报。隧道接收端的 IPv6 结点最终收到该 IPv4 数据报（它是该 IPv4 数据报的目的地！），并确定该 IPv4 数据报含有一个 IPv6 数据报，于是从中取出 IPv6 数据报，然后再为该 IPv6 数据报提供路由，就好像它是从一个直接相连的 IPv6 邻居那里接收到该 IPv6 数据报的一样。

在结束本节前需要说明的是，尽管 IPv6 的采用最初表现为一个缓慢启动的过程 [Lawton 2001]，但是最近已经积蓄了力量。有关 2008 年 IPv6 部署的讨论参见 [Huston 2008b]；有关美国 IPv6 部署的概况参见 [NIST IPv6 2012]。大量增加的设备如 IP 使能电话与其他便携式设备为 IPv6 的更广泛部署提供了额外的推动力。欧洲的第三代合作计划 [3GPP 2012] 已规定了 IPv6 为移动多媒体的标准编址方案。

我们能从 IPv6 经验中学到的重要一课是，要改变网络层协议是极其困难的。自从 20 世纪 90 年代早期以来，有许多新的网络层协议被鼓吹为因特网的下一次重大革命，但这

些协议中的大多数至今为止只取得了有限突破。这些协议包括 IPv6、多播协议（4.7 节）、资源预留协议（第 7 章）。在网络层中引入新的协议的确如同替换一幢房子的基石，即在不拆掉整幢房子（或至少临时重新安置房屋住户）的情况下是很难完成上述工作的。在另一方面，因特网却已见证了在应用层中新协议的快速部署。典型的例子当然有 Web、即时讯息、P2P 文件共享。其他例子包括音频与视频流和分布式游戏。引入新的应用层协议就像给一幢房子重新刷一层漆，这是相对容易做的事，如果你选择了一个好看的颜色，邻居将会照搬你的选择。总之，我们在未来能够期待看到因特网网络层中的这些变化，但这些变化出现的时间尺度很可能要比在应用层中出现的变化要慢得多。

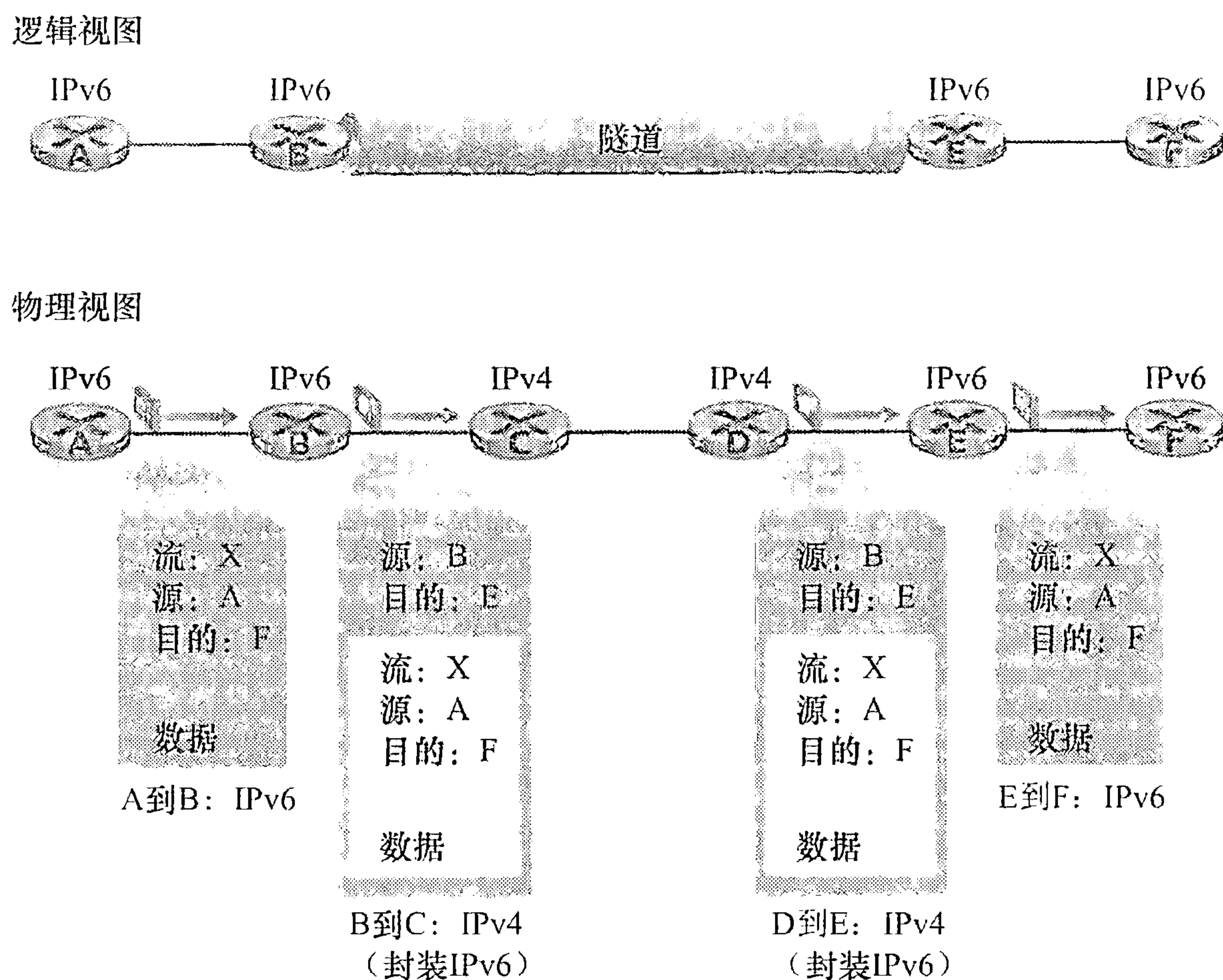


图 4-26 建隧道

4.4.5 涉足 IP 安全性

4.4.3 节详细讨论了 IPv4，包括它提供的服务以及如何实现这些服务的方法。在阅读该节内容的时候，你可能注意到了并没有提到任何安全服务。IPv4 的设计的确是在因特网主要用于相互信任的联网研究人员之间的时代（20 世纪 70 年代）。创建一个能够集成多种链路层技术的计算机网络已经面临了足够多的挑战了，无法顾及安全性了。

但是，在安全性成了主要关注对象的今天，因特网研究人员已经在继续前进了，他们设计了提供各种安全性服务的新型网络层协议。这些协议之一是 IPsec，这是一种非常流行的安全网络层协议，也在虚拟专用网（VPN）中得到了广泛部署。尽管在第 8 章中我们将更为详细地讨论 IPsec 及其密码学基础，但在本节中我们只对 IPsec 服务提供一个简要的、高层的概述。

IPsec 已被设计为与 IPv4 和 IPv6 向后兼容。特别是，为了享受 IPsec 带来的好处，我们不需要替换因特网中的所有路由器和主机中的协议栈。例如，使用运输模式（两种 IPsec “模式” 之一）时，如果两台主机要安全地通信，IPsec 仅需要在这两台主机中可用。所有其他路由器和主机能够继续运行普通的 IPv4。

为了具体起见，我们这里将关注 IPsec 的运输模式。使用这种模式，两台主机首先在它们之间创建一个 IPsec 会话。（因此 IPsec 是面向连接的！）使用适当的会话，在这两台主机之间发送的所有 TCP 和 UDP 报文段都享受 IPsec 提供的安全性服务。在发送端，运输层向 IPsec 传递一个报文段。IPsec 然后加密该报文段，在报文段上添加附加的安全性字段，并且在一个普通的 IP 数据报中封装得到的有效载荷。（实际中比上述过程要复杂一点，我们将在第 8 章详细讨论。）发送主机接下来向因特网中发送数据报，因特网则将数据报传送到目的主机。在那里，IPsec 解密报文段并将脱密的报文段传送给运输层。

由 IPsec 会话提供的服务包括：

- 密码技术约定。这种机制允许两台通信的主机对加密算法和密钥达成一致。
- IP 数据报有效载荷的加密。当发送主机从运输层接收到一个报文段时，IPsec 加密该有效载荷。该有效载荷仅能由在接收主机中的 IPsec 解密。
- 数据完整性。IPsec 允许接收主机验证数据报的首部字段，保证被加密的有效载荷在其数据报从源到目的地的路由器中传输时没有被修改过。
- 初始鉴别。当一台主机从某受信任的源（具有一个受信任的密钥，参见第 8 章）接收到一个 IPsec 数据报时，该主机确信在数据报中的源 IP 地址是该数据报的实际源。

当两台主机在它们之间创建了一个 IPsec 会话时，在它们之间发送的所有 TCP 和 UDP 报文段将被加密和鉴别。IPsec 因此提供了地毯式覆盖，使这两台主机之间的所有网络应用进行安全通信。

通过使用 IPsec，一个公司能够在非安全的公共因特网中进行安全通信。为了进行说明，我们这里只看一个简单的例子。考虑一个拥有大批销售人员的公司，这些销售人员分布在各地跑业务。假定各种销售人员需要经常查询公司的敏感信息（例如价格和产品信息），这些信息存储在公司总部的一台服务器上。进一步假设销售人员也需要彼此发送敏感文档。使用 IPsec 怎样能够做到这一点呢？如你猜想的那样，我们在这台服务器和所有销售人员的便携机上安装 IPsec。借助于安装在这些主机上的 IPsec，某销售人员无论何时需要与服务器通信或与另一名销售人员通信，这些通信会话将是安全的。

4.5 路由选择算法

到目前为止，我们在本章中主要研究了网络层的转发功能。我们知道当分组到达一台路由器时，该路由器索引其转发表并决定该分组被指向的链路接口。我们也知道路由选择算法在网络路由器中运行、交换和计算信息，用这些信息配置这些转发表。路由选择算法和转发表之间的相互影响如图 4-2 所示。在已经较为深入地研究了转发后，我们将注意力转向本章的其他重要主题，即网络层的至关重要的路由选择功能。不管网络层提供的是数据报服务（在此情况下，在给定源和目的地址之间的不同分组可能采用不同的路由），还是虚电路服务（在此情况下，在给定源和目的地址之间的所有分组将采用相同路径），网络层都必须为从发送方到接收方的分组确定所采用的路径。我们将看到路由选择的工作是：确定从发送方到接收方通过路由器网络的好路径（等价为路由）。

主机通常直接与一台路由器相连接，该路由器即为该主机的默认路由器（default router），又称为该主机的第一跳路由器（first-hop router）。每当主机发送一个分组时，该分组被传

送给它的默认路由器。我们将源主机的默认路由器称作**源路由器**（source router），把目的主机的默认路由器称作**目的路由器**（destination router）。一个分组从源主机到目的主机的路由选择问题显然可归结为从源路由器到目的路由器的路由选择问题。这是本节的重点。

因此，路由选择算法的目的是简单的：给定一组路由器以及连接路由器的链路，路由选择算法要找到一条从源路由器到目的路由器的“好”路径。通常，一条好路径指具有最低费用的路径。然而我们将看到，实践中现实世界还关心诸如策略之类的问题（例如，诸如“属于组织 Y 的路由器 X 不应转发任何来源于组织 Z 网络的分组”之类的规则），这也使得概念简单、性能优秀的算法变得复杂。然而这些概念简单、性能优秀的算法的理论奠定了当今网络路由选择实践的基础。

可以用图来形式化描述路由选择问题。我们知道图（graph） $G = (N, E)$ 是一个 N 个结点和 E 条边的集合，其中每条边是取自 N 的一对结点。在网络层路由选择的环境中，图中的结点表示路由器，这是做出分组转发决定的点；连接这些结点的边表示这些路由器之间的物理链路。这样一个计算机网络图的抽象显示在图 4-27 中。若要查看某些表示实际网络图的图，参见 [Dodge 2012; Cheswick 2000]；对于基于不同的图模型建模因特网的好坏的讨论，参见 [Zegura 1997; Faloutsos 1999; Li 2004]。

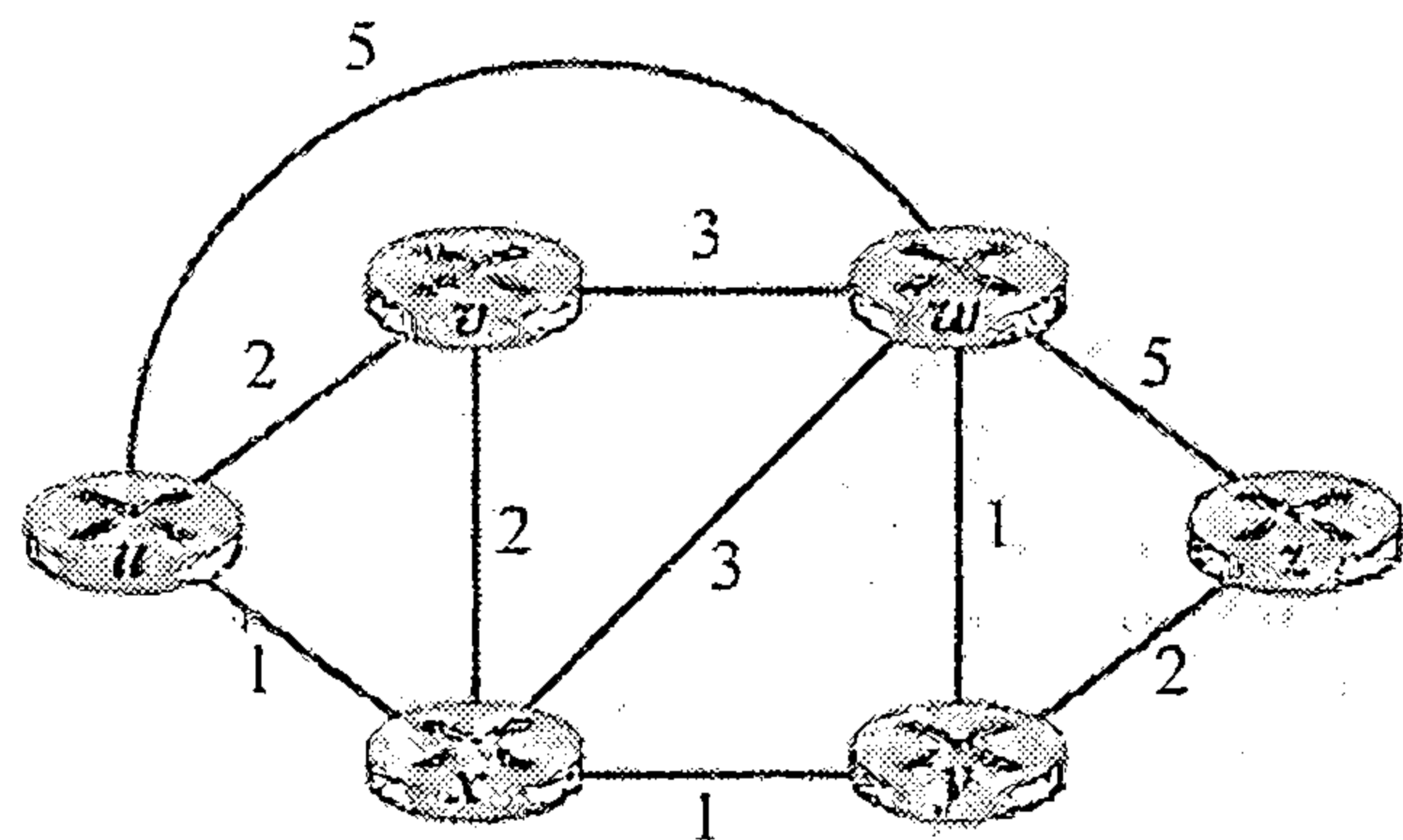


图 4-27 一个计算机网络的抽象图模型

如图 4-27 所示，一条边还有一个值表示它的费用。通常，一条边的费用可反映出对应链路的物理长度（例如一条越洋链路的费用可能比一条短途陆地链路的费用高）、链路速度，或与该链路相关的金融上的费用。为了我们的目的，我们只将这些链路费用看成是给定的，而不必操心这些值是如何确定的。对于 E 中的任一条边 (x, y) ，我们用 $c(x, y)$ 表示结点 x 和 y 间边的费用。如果不属于 E 的结点对 (x, y) ，置 $c(x, y) = \infty$ 。此外，我们考虑都是无向图（即图的边没有方向），因此边 (x, y) 与边 (y, x) 是相同的并且 $c(x, y) = c(y, x)$ 。如果 (x, y) 属于 E ，结点 y 也被称为结点 x 的**邻居**（neighbor）。

在图抽象中为各条边指派了费用后，路由选择算法的自然目标是找出从源到目的地间的最低费用路径。为了使问题更为精确，回想在图 $G = (N, E)$ 中的一条**路径**（path）是一个结点的序列 $(x_1, x_2, \dots, x_p)$ ，这样每一个对 $(x_1, x_2), (x_2, x_3), \dots, (x_{p-1}, x_p)$ 是 E 中的边。路径 $(x_1, x_2, \dots, x_p)$ 的费用只是沿着路径所有边的费用的总和，即 $c(x_1, x_2) + c(x_2, x_3) + \dots + c(x_{p-1}, x_p)$ 。给定任何两个结点 x 和 y ，通常在这两个结点之间有许多条路径，每条路径都有一个费用。这些路径中的一条或多条是**最低费用路径**（least-cost path）。因此最低费用路径问题是清楚的：找出源和目的地之间的具有最低费用的一条路。例如，在图 4-27 中，源结点 u 和目的结点 w 之间的最低费用路径是 (u, x, y, w) ，具有的路径费用是 3。注意到若在图中的所有边具有相同的费用，则最低费用路径也就是**最短路径**（shortest path），即在源和目的地之间的具有最少链路数量的路径。

作为一个简单练习，试找出图 4-27 中从结点 u 到结点 z 的最低费用路径，并要反映出你是如何算出该路径的。如果你像大多数人一样，通过考察图 4-27，跟踪几条从 u 到 z 的路由，你就能找出从 u 到 z 的路径，然后以某种方式来确信你所选择的路径就是所有可能

的路径中具有最低费用的路径。(你考察过 u 到 z 之间的所有 17 条可能的路径吗? 很可能没有!) 这种计算就是一种集中式路由选择算法的例子, 即路由选择算法在一个位置 (即你的大脑中) 运行, 具有该网络的完整信息。对路由选择算法的一种广义分类方式是根据该算法是全局式的还是分散式的来加以区分。

- **全局式路由选择算法** (global routing algorithm) 用完整的、全局性的网络知识计算出从源到目的地之间的最低费用路径。也就是说, 该算法以所有结点之间的连通性及所有链路费用为输入。这就要求该算法在真正开始计算以前, 要以某种方式获得这些信息。计算本身可在某个场点 (集中式全局路由选择算法) 进行, 或可在多个场点重复进行。然而这里的主要区别在于, 全局式算法具有关于连通性和链路费用方面的完整信息。实践中, 具有全局状态信息的算法常被称作**链路状态** (Link State, LS) 算法, 因为该算法必须知道网络中每条链路的费用。我们将在 4.5.1 节中学习 LS 算法。
- **分散式路由选择算法** (decentralized routing algorithm) 以迭代、分布式的方式计算出最低费用路径。没有结点拥有关于所有网络链路费用的完整信息, 而每个结点仅有与其直接相连链路的费用知识即可开始工作。然后, 通过迭代计算过程并与相邻结点 (即与该结点相连链路的另一端的结点) 交换信息, 一个结点逐渐计算出到达某目的结点或一组目的结点的最低费用路径。我们将在后面的 4.5.2 节学习一个称为**距离向量** (Distance-Vector, DV) 算法的分散式路由选择算法。之所以叫做 DV 算法, 是因为每个结点维护到网络中所有其他结点的费用 (距离) 估计的向量。

路由选择算法的第二种广义分类方式是根据算法是静态的还是动态的进行分类。在**静态路由选择算法** (static routing algorithm) 中, 随着时间的流逝, 路由的变化是非常缓慢的, 通常是人工干预进行调整 (如人为手工编辑一台路由器的转发表)。**动态路由选择算法** (dynamic routing algorithm) 能够当网络流量负载或拓扑发生变化时改变路由选择路径。一个动态算法可周期性地运行或直接响应拓扑或链路费用的变化而运行。虽然动态算法易于对网络的变化做出反应, 但也更容易受诸如路由选择循环、路由振荡之类问题的影响。

路由选择算法的第三种分类方式是根据它是负载敏感的还是负载迟钝的进行划分。在**负载敏感算法** (load-sensitive algorithm) 中, 链路费用会动态地变化以反映出底层链路的当前拥塞水平。如果当前拥塞的一条链路与高费用相联系, 则路由选择算法趋向于避开该拥塞链路来选择路由。而早期的 ARPAnet 路由选择算法就是负载敏感的 [McQuillan 1980], 所以遇到了许多难题 [Huitema 1998]。当今的因特网路由选择算法 (如 RIP、OSPF 和 BGP) 都是**负载迟钝的** (load-insensitive), 因为某条链路的费用不明显地反映其当前 (或最近) 的拥塞水平。

4.5.1 链路状态路由选择算法

前面讲过, 在链路状态算法中, 网络拓扑和所有的链路费用都是已知的, 也就是说可用作 LS 算法的输入。实践中这是通过让每个结点向网络中所有其他结点广播链路状态分组来完成的, 其中每个链路状态分组包含它所连接的链路的特征和费用。在实践中 (例如使用因特网的 OSPF 路由选择协议, 讨论见 4.6.1 节), 这经常由**链路状态广播** (link state broadcast) 算法 [Perlman 1999] 来完成。我们将在 4.7 节中讨论广播算法。结点广播的

结果是所有结点具有了该网络的等同的、完整的视图。于是每个结点都能够像其他结点一样，运行 LS 算法并计算出相同的最低费用路径集合。

我们下面给出的链路状态路由选择算法叫做 Dijkstra 算法，该算法以其发明者命名。一个密切相关的算法是 Prim 算法，有关图算法的一般性讨论参见 [Cormen 2001]。Dijkstra 算法计算从某结点（源结点，我们称之为 u ）到网络中所有其他结点的最低费用路径。Dijkstra 算法是迭代算法，其性质是经算法的第 k 次迭代后，可知道到 k 个目的结点的最低费用路径，在到所有目的结点的最低费用路径之中，这 k 条路径具有 k 个最低费用。我们定义下列记号。

- $D(v)$ ：到算法的本次迭代，从源结点到目的结点 v 的最低费用路径的费用。
- $p(v)$ ：从源到 v 沿着当前最低费用路径的前一结点（ v 的邻居）。
- N' ：结点子集；如果从源到 v 的最低费用路径已确知， v 在 N' 中。

该全局路由选择算法由一个初始化步骤和其后的循环组成。循环执行的次数与网络中结点个数相同。一旦终止，该算法就计算出了从源结点 u 到网络中每个其他结点的最短路径。

源结点 u 的链路状态（LS）算法

```
1 Initialization:
2    $N' = \{u\}$ 
3   for all nodes  $v$ 
4     if  $v$  is a neighbor of  $u$ 
5       then  $D(v) = c(u,v)$ 
6     else  $D(v) = \infty$ 
7
8 Loop
9   find  $w$  not in  $N'$  such that  $D(w)$  is a minimum
10  add  $w$  to  $N'$ 
11  update  $D(v)$  for each neighbor  $v$  of  $w$  and not in  $N'$ :
12     $D(v) = \min( D(v), D(w) + c(w,v) )$ 
13  /* new cost to  $v$  is either old cost to  $v$  or known
14     least path cost to  $w$  plus cost from  $w$  to  $v$  */
15 until  $N' = N$ 
```

举一个例子，考虑图 4-27 中的网络，计算从 u 到所有可能目的地的最低费用路径。该算法的计算过程以表格方式总结于表 4-3 中，表中的每一行给出了迭代结束时该算法的变量的值。我们详细地考虑前几个步骤。

表 4-3 在图 4-27 中的网络上运行的链路状态算法

步骤	N'	$D(v), p(v)$	$D(w), p(w)$	$D(x), p(x)$	$D(y), p(y)$	$D(z), p(z)$
0	u	2, u	5, u	1, u	∞	∞
1	ux	2, u	4, x		2, x	∞
2	uxy	2, u	3, y			4, y
3	$uxyv$		3, y			4, y
4	$uxyvw$					4, y
5	$uxyvwz$					

- 在初始化步骤，从 u 到与其直接相连的邻居 v 、 x 、 w 的当前已知最低费用路径分别初始化为 2、1 和 5。特别值得注意的是，到 w 的费用被设为 5（尽管我们很快就会看见一条费用更小的路径确实存在），因为这是从 u 到 w 的直接（一跳）链路费用。到 y 与 z 的费用被设为无穷大，因为它们不直接与 u 连接。

- 在第一次迭代中，我们观察那些还未加到集合 N' 中的结点，并且找出在前一次迭代结束时具有最低费用的结点。那个结点便是 x ，其费用是 1，因此 x 被加到集合 N' 中。于是 LS 算法中的第 12 行中的程序被执行，以更新所有结点 v 的 $D(v)$ ，产生表 4-3 中第 2 行（步骤 1）所示的结果。到 v 的路径费用未变。经过结点 x 到 w （在初始化结束时其费用为 5）的路径费用被发现为 4。因此这条具有更低费用的路径被选中，且沿从 u 开始的最短路径上 w' 的前一结点被设为 x 。类似地，到 y （经过 x ）的费用被计算为 2，且该表也被相应地更新。
- 在第二次迭代时，结点 v 与 y 被发现具有最低费用路径（2），并且我们任意改变次序将 y 加到集合 N' 中，使得 N' 中含有 u 、 x 和 y 。到仍不在 N' 中的其余结点（即结点 v 、 w 和 z ）的费用通过 LS 算法中的第 12 行进行更新，产生如表 4-3 中第 3 行所示的结果。
- 如此等等。

当 LS 算法终止时，对于每个结点，我们都得到从源结点沿着它的最低费用路径的前一结点。对于每个前一结点，我们又有它的前一结点，以此方式我们可以构建从源结点到所有目的结点的完整路径。通过对每个目的结点存放从 u 到目的地的最低费用路径上的下一跳结点，在一个结点（如结点 u ）中的转发表则能够根据此信息而构建。图 4-28 显示了对于图 4-27 中的网络产生的最低费用路径和 u 中的转发表。

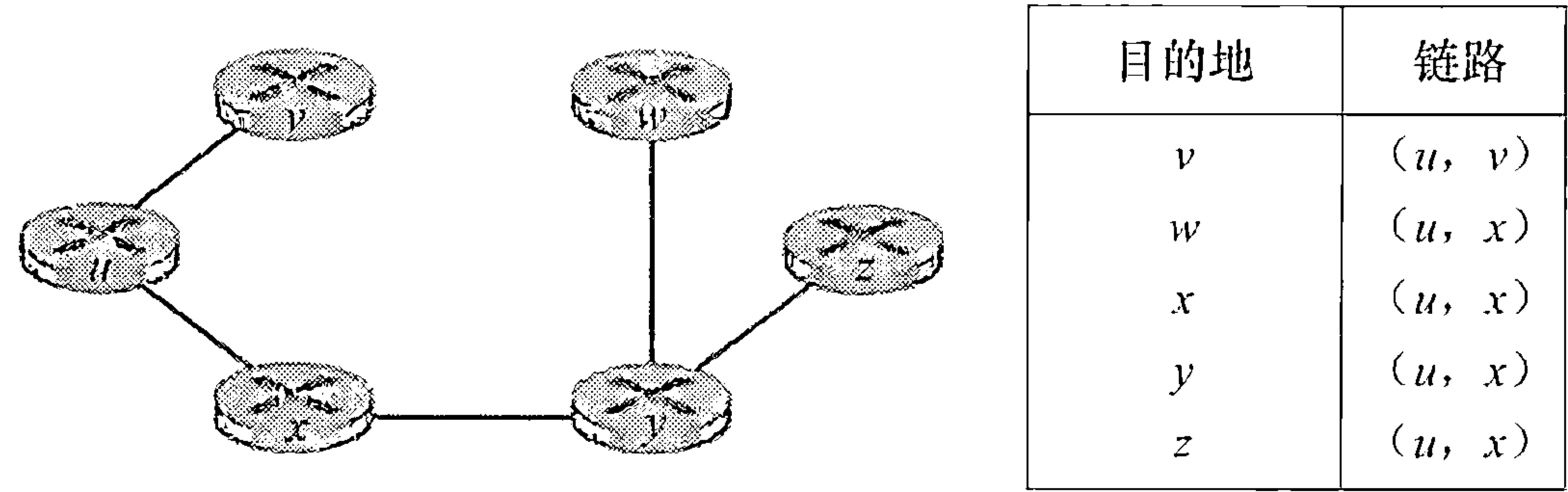


图 4-28 对于结点 u 的最低费用路径和转发表

该算法的计算复杂性是什么？即给定 n 个结点（不算源结点），在最坏情况下要经过多少次计算，才能找到从源结点到所有目的结点的最低费用路径？在第一次迭代中，我们需要搜索所有的 n 个结点以确定出结点 w —— w 不在 N' 中且具有最低费用。在第二次迭代时，我们需要检查 $n - 1$ 个结点以确定最低费用。第三次对 $n - 2$ 个结点迭代，依次类推。总之，我们在所有迭代中需要搜寻的结点总数为 $n(n + 1)/2$ ，因此我们说前面实现的链路状态算法在最差情况下复杂性为 $O(n^2)$ 。（该算法的一种更复杂的实现是使用一种称为堆的数据结构，能用对数时间而不是线性时间得到第 9 行的最小值，因此减少其复杂性。）

在完成 LS 算法的讨论之前，我们考虑一下可能出现的问题。图 4-29 显示了一个简单的网络拓扑，图中的链路费用等于链路上承载的负载，例如反映要历经的时延。在该例中，链路费用是非对称的，即仅当在链路 (u, v) 两个方向所承载的负载相同时 $c(u, v)$ 与 $c(v, u)$ 才相等。在该例中，结点 z 产生发往 w 的一个单元的流量，结点 x 也产生发往 w 的一个单元的流量，并且结点 y 也产生发往 w 的一个数量为 e 的流量。初始路由选择情况如图 4-29a 所示，其链路费用对应于承载的流量。

当 LS 算法再次运行时，结点 y 确定（基于图 4-29a 所示的链路费用）顺时针到 w 的路径费用为 1，而逆时针到 w 的路径费用（一直使用的）是 $1 + e$ 。因此 y 到 w 的最低费用

路径现在是顺时针的。类似地, x 确定其到 w 的新的最低费用路径也是顺时针的, 产生如图 4-29b 中所示的费用。当 LS 算法下次运行时, 结点 x 、 y 和 z 都检测到一条至 w 的逆时针方向零费用路径, 它们都将其流量引导到逆时针方向的路由上。下次 LS 算法运行时, x 、 y 和 z 都将其流量引导到顺时针方向的路由上。

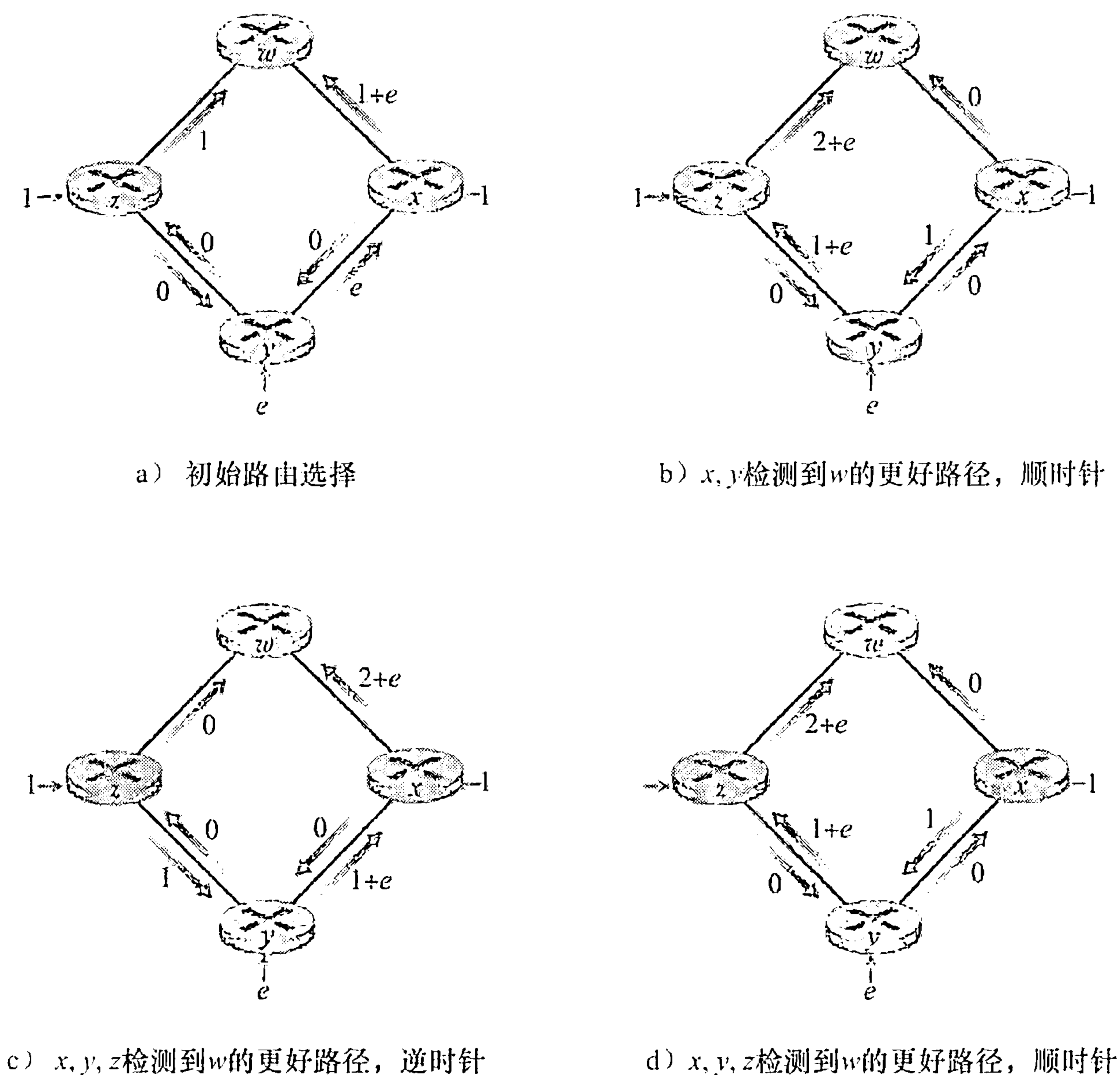


图 4-29 拥塞敏感的路由选择的振荡

如何才能防止这样的振荡 (它不只是出现在链路状态算法中, 而且也可能出现在任何使用拥塞或基于时延的链路测度的算法中)。一种解决方案可能强制链路费用不依赖于所承载的流量, 但那是一种不可接受的解决方案, 因为路由选择的目标之一就是要避开高度拥塞 (如高时延) 的链路。另一种解决方案就是确保并非所有的路由器都同时运行 LS 算法。这似乎是一个更合理的方案, 因为我们希望即使路由器以相同周期运行 LS 算法, 在每台路由器上算法执行的时机也将是不同的。有趣的是, 研究人员近来已注意到了因特网上的路由器能在它们之间进行自同步 [Floyd Synchronization 1994]。这就是说, 即使它们初始时以同一周期但在不同时刻执行算法, 算法执行时机最终会在路由器上变为同步并保持之。避免这种自同步的一种方法是, 让每台路由器发送链路通告的时间随机化。

学习过 LS 算法之后, 我们接下来考虑目前在实践中使用的其他重要的路由选择算法, 即距离向量路由选择算法。

4.5.2 距离向量路由选择算法

距离向量 (Distance-Vector, DV) 算法是一种迭代的、异步的和分布式的算法, 而 LS

算法是一种使用全局信息的算法。说它是分布式的，是因为每个结点都要从一个或多个直接相连邻居接收某些信息，执行计算，然后将其计算结果分发给邻居。说它是迭代的，是因为此过程一直要持续到邻居之间无更多信息要交换为止。（有趣的是，此算法是自我终止的，即没有计算应该停止的信号，它就停止了。）说它是异步的，是因为它不要求所有结点相互之间步伐一致地操作。我们将看到一个异步的、迭代的、自我终止的、分布式的算法比一个集中式的算法要有趣得多。

在我们给出 DV 算法之前，有必要讨论一下存在于最低费用路径的费用之间的一种重要关系。令 $d_x(y)$ 是从结点 x 到结点 y 的最低费用路径的费用。则该最低费用与著名的 Bellman-Ford 方程相关，即

$$d_x(y) = \min_v \{c(x, v) + d_v(y)\} \quad (4-1)$$

方程中的 $\min_v$ 是对于 x 的所有邻居的。Bellman-Ford 方程是相当直观的。实际上，从 x 到 v 遍历之后，如果我们接下来取从 v 到 y 的最低费用路径，则该路径费用将是 $c(x, v) + d_v(y)$ 。因此我们必须通过遍历某些邻居 v 开始，从 x 到 y 的最低费用是对所有邻居 v 的 $c(x, v) + d_v(y)$ 的最小值。

但是对于那些可能怀疑该方程正确性的人，我们核查在图 4-27 中的源结点 u 和目的结点 z 。源结点 u 有 3 个邻居：结点 v 、 x 和 w 。通过遍历该图中的各条路径，容易看出 $d_v(z) = 5$ 、 $d_x(z) = 3$ 和 $d_w(z) = 3$ 。将这些值连同费用 $c(u, v) = 2$ 、 $c(u, x) = 1$ 和 $c(u, w) = 5$ 代入方程 (4-1)，得出 $d_u(z) = \min\{2 + 5, 5 + 3, 1 + 3\} = 4$ ，这显然是正确的，并且对同一个网络来说，这正是 Dijkstra 算法为我们提供的结果。这种快速验证应当有助于消除你可能具有的任何怀疑。

Bellman-Ford 方程不止是一种智力上的珍品，它实际上具有重大的实践重要性。特别是对 Bellman-Ford 方程的解为结点 x 的转发表提供了表项。为了理解这一点，令 v^* 是取得方程 (4-1) 中最小值的任何相邻结点。接下来，如果结点 x 要沿着最低费用路径向结点 y 发送一个分组，它应当首先向结点 v^* 转发该分组。因此，结点 x 的转发表将指定结点 v^* 作为最终目的地 y 的下一跳路由器。Bellman-Ford 方程的另一个重要实际贡献是它提出了将在 DV 算法中发生的邻居到邻居通信的形式。

其基本思想如下。每个结点 x 以 $D_x(y)$ 开始，对在 N 中的所有结点，估计从它自己到结点 y 的最低费用路径的费用。令 $\mathbf{D}_x = [D_x(y) : y \in N]$ 是结点 x 的距离向量，该向量是从 x 到在 N 中的所有其他结点 y 的费用估计的向量。使用 DV 算法，每个结点 x 维护下列路由选择信息：

- 对于每个邻居 v ，从 x 到直接相连邻居 v 的费用为 $c(x, v)$ 。
- 结点 x 的距离向量，即 $\mathbf{D}_x = [D_x(y) : y \in N]$ ，包含了 x 到 N 中所有目的地 y 的费用的估计值。
- 它的每个邻居的距离向量，即对 x 的每个邻居 v ，有 $\mathbf{D}_v = [D_v(y) : y \in N]$ 。

在该分布式、异步算法中，每个结点不时地向它的每个邻居发送它的距离向量副本。当结点 x 从它的任何一个邻居 v 接收到一个新距离向量，它保存 v 的距离向量，然后使用 Bellman-Ford 方程更新它自己的距离向量如下：

$$D_x(y) = \min_v \{c(x, v) + D_v(y)\} \quad \text{对 } N \text{ 中的每个结点}$$

如果结点 x 的距离向量因这个更新步骤而改变，结点 x 接下来将向它的每个邻居发送其更新后的距离向量。令人惊奇的是，只要所有的结点继续以异步方式交换它们的距离向

量，每个费用估计 $D_x(y)$ 收敛到 $d_x(y)$ ， $d_x(y)$ 为从结点 x 到结点 y 的实际最低费用路径的费用 [Bersekas 1991]！

距离向量 (DV) 算法

在每个结点 x ：

```

1  Initialization:
2    for all destinations y in N:
3       $D_x(y) = c(x,y)$  /* if y is not a neighbor then  $c(x,y) = \infty$  */
4    for each neighbor w
5       $D_w(y) = ?$  for all destinations y in N
6    for each neighbor w
7      send distance vector  $D_x = [D_x(y): y \text{ in } N]$  to w
8
9  loop
10   wait (until I see a link cost change to some neighbor w or
11         until I receive a distance vector from some neighbor w)
12
13   for each y in N:
14      $D_x(y) = \min_v \{c(x,v) + D_v(y)\}$ 
15
16   if  $D_x(y)$  changed for any destination y
17     send distance vector  $D_x = [D_x(y): y \text{ in } N]$  to all neighbors
18
19 forever

```

在该 DV 算法中，当结点 x 发现它的直接相连的链路费用变化或从某个邻居接收到一个距离向量的更新时，它就更新其距离向量估计。但是为了一个给定的目的地 y 而更新它的转发表，结点 x 真正需要知道的不是到 y 的最短路径距离，而是沿着最短路径到 y 的下一跳路由器邻居结点 $v^*(y)$ 。如你可能期望的那样，下一跳路由器 $v^*(y)$ 是在 DV 算法第 14 行中取得最小的邻居 v 。（如果有多个取得最小的邻居 v ，则 $v^*(y)$ 能够是其中任何一个最小的邻居。）因此，对于每个目的地 y ，在第 13 ~ 14 行中，结点 x 也决定 $v^*(y)$ 并更新它对目的地 y 的转发表。

前面讲过 LS 算法是一种全局算法，在于它要求每个结点在运行 Dijkstra 算法之前，首先获得该网络的完整信息。DV 算法是分布式的，它不使用这样的全局信息。实际上，结点具有的唯一信息是它到直接相连邻居的链路费用和它从这些邻居接收到的信息。每个结点等待来自任何邻居的更新（第 10 ~ 11 行），当接收到一个更新时计算它的新距离向量（第 14 行）并向它的邻居分布其新距离向量（第 16 ~ 17 行）。许多类似 DV 的算法在实践中被用于多种路由选择协议中，包括因特网的 RIP 和 BGP、ISO IDRP、Novell IPX 和早期的 ARPAnet。

图 4-30 举例说明了 DV 算法的运行，应用场合是该图顶部有三个结点的简单网络。算法操作以同步的方式显示出来，其中所有结点同时从其邻居接收报文，计算其新距离向量，如果距离向量发生了变化则通知其邻居。学习完这个例子后，你应认识到算法以异步方式也能正确运行，异步方式中可在任意时刻出现结点计算与更新的产生/接收。

该图最左边一列显示了这 3 个结点各自的初始路由选择表（routing table）。例如，位于左上角的表是结点 x 的初始路由选择表。在一张特定的路由选择表中，每行是一个距离向量——特别是每个结点的路由选择表包括了它的距离向量和它的每个邻居的距离向量。因此，在结点 x 的初始路由选择表中的第一行是 $D_x = [D_x(x), D_x(y), D_x(z)] = [0, 2, 7]$ 。在该表的第二和第三行是最近分别从结点 y 和 z 收到的距离向量。因为在初始化时结点 x 还没有从结点 y 和 z 收到任何东西，所以第二行和第三行表项中被初始化为无穷大。

初始化后，每个结点向它的两个邻居发送其距离向量。图 4-30 中用从表的第一列到表的第二列的箭头说明了这一情况。例如，结点 x 向两个结点 y 和 z 发送了它的距离向量 $D_x = [0, 2, 7]$ 。在接收到该更新后，每个结点重新计算它自己的距离向量。例如，结点 x 计算

$$D_x(x) = 0$$
$$D_x(y) = \min\{c(x,y) + D_y(y), c(x,z) + D_z(y)\} = \min\{2 + 0, 7 + 1\} = 2$$
$$D_x(z) = \min\{c(x,y) + D_y(z), c(x,z) + D_z(z)\} = \min\{2 + 1, 7 + 0\} = 3$$

第二列因此为每个结点显示了结点的新距离向量连同刚从它的邻居接收到的距离向量。注意到，例如结点 x 到结点 z 的最低费用估计 $D_x(z)$ 已经从 7 变成了 3。还应注意到，对于结点 x ，结点 y 在该 DV 算法的第 14 行中取得了最小值；因此在该算法的这个阶段，我们在结点 x 得到了 $v^*(y) = y$ 和 $v^*(z) = y$ 。

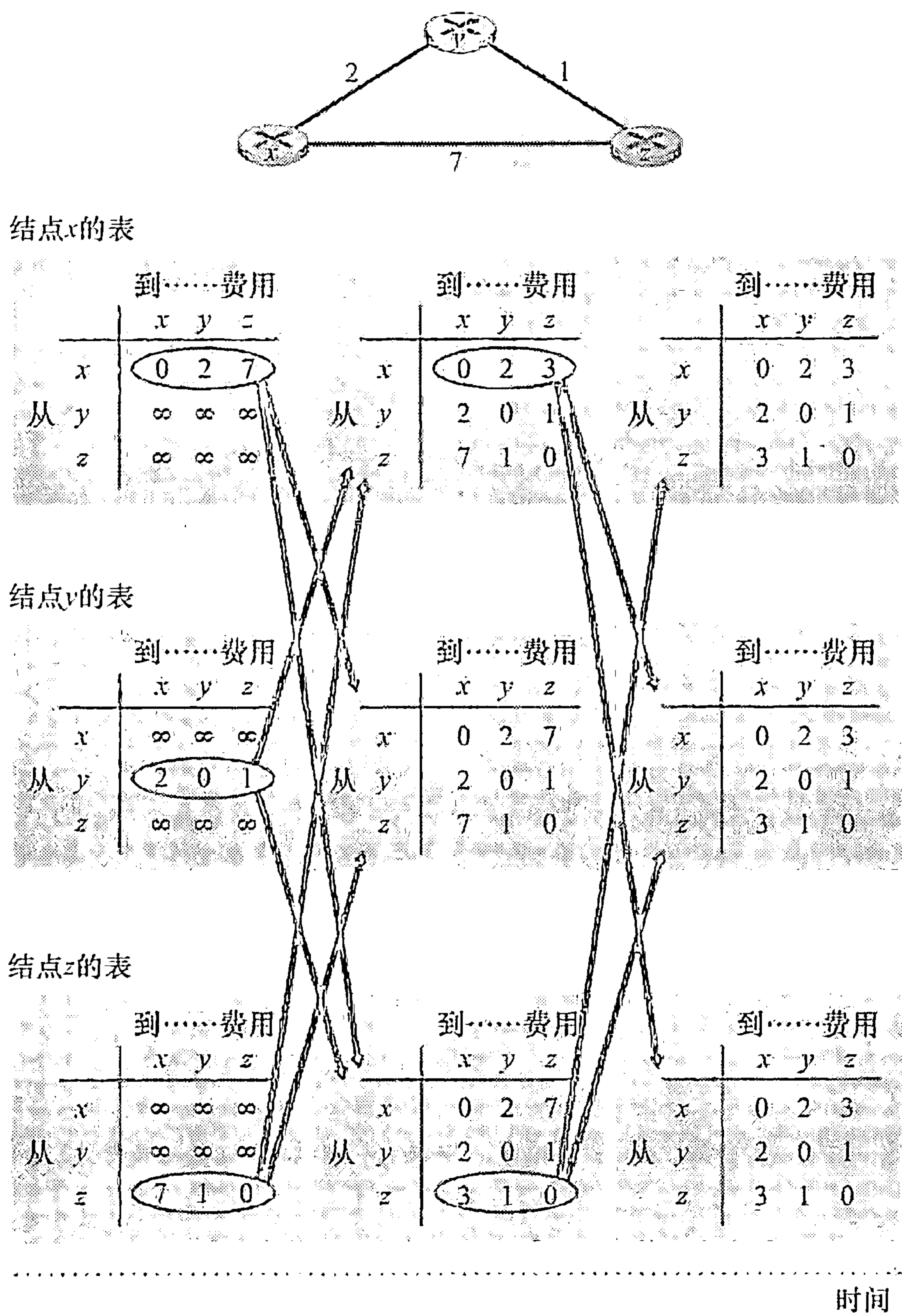


图 4-30 距离向量 (DV) 算法

在结点重新计算它们的距离向量之后，它们再次向其邻居发送它们的更新距离向量（如果它们已经改变的话）。图 4-30 中由从表第二列到表第三列的箭头说明了这一情况。注意到仅有结点 x 和结点 z 发送了更新：结点 y 的距离向量没有发生变化，因此结点 y 没有发送更新。在接收到这些更新后，这些结点则重新计算它们的距离向量并更新它们的路

由选择表，这些显示在第三列中。

从邻居接收更新距离向量、重新计算路由选择表项和通知邻居到目的地的最低费用路径的费用已经变化的过程继续下去，直到无更新报文发送为止。在这个时候，因为无更新报文发送，将不会出现进一步的路由选择表计算，该算法将进入静止状态；即所有的结点将执行 DV 算法的第 10 ~ 11 行的等待。该算法停留在静止状态，直到一条链路费用发生改变，如下面所讨论的那样。

1. 距离向量算法：链路费用改变与链路故障

当一个运行 DV 算法的结点检测到从它自己到邻居的链路费用发生变化时（第 10 ~ 11 行），它就更新其距离向量（第 13 ~ 14 行），并且如果最低费用路径的费用发生了变化，向邻居通知其新的距离向量（第 16 ~ 17 行）。图 4-31a 图示了从 y 到 x 的链路费用从 4 变为 1 情况。我们在此只关注 y 与 z 到目的地 x 的距离表中的有关表项。该 DV 算法导致下列事件序列的出现：

- 在 t_0 时刻， y 检测到链路费用变化（费用从 4 变为 1），更新其距离向量，并通知其邻居这个变化，因为最低费用路径的费用已改变。
- 在 t_1 时刻， z 收到来自 y 的更新报文并更新了其距离表。它计算出到 x 的新最低费用（从费用 5 减为费用 2），它向其邻居发送了它的新距离向量。
- 在 t_2 时刻， y 收到来自 z 的更新并更新其距离表。 y 的最低费用未变，因此 y 不发送任何报文给 z 。该算法进入静止状态。

因此，对于该 DV 算法只需两次迭代就到达了静止状态。在 x 与 y 之间费用减少的好消息通过网络得到了迅速传播。

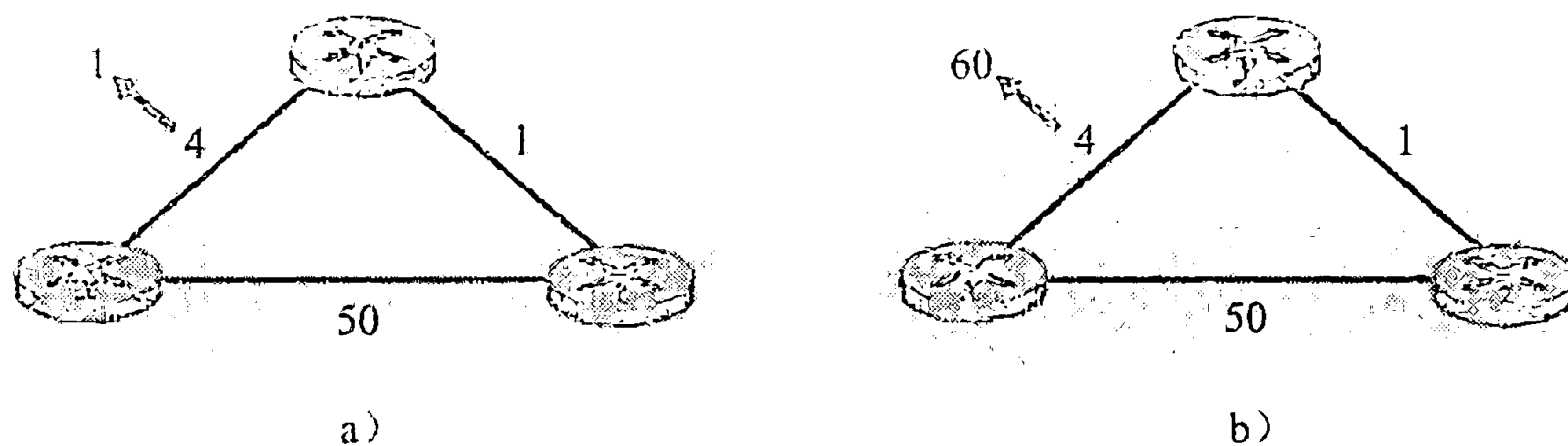


图 4-31 链路费用改变

我们现在考虑一下当某链路费用增加时发生的情况。假设 x 与 y 之间的链路费用从 4 增加到 60，如图 4-31b 所示。

1) 在链路费用变化之前， $D_y(x) = 4$ ， $D_y(z) = 1$ ， $D_z(y) = 1$ 和 $D_z(x) = 5$ 。在 t_0 时刻， y 检测到链路费用变化（费用从 4 变为 60）。 y 计算其到 x 的新的最低费用路径的费用值为

$$D_y(x) = \min\{c(y,x) + D_x(x), c(y,z) + D_z(x)\} = \min\{60 + 0, 1 + 5\} = 6$$

当然，从网络全局的视角来看，我们能够看出经过 z 的这个新费用是错误的。但结点 y 仅有的信息是：它到 x 的直接费用是 60，且 z 上次已告诉 y ， z 能以费用 5 到 x 。因此，为了到达 x ， y 将通过 z 路由，完全期望 z 能以费用 5 到达 x 。到了 t_1 时刻，我们遇到路由选择环路（routing loop），即为到达 x ， y 通过 z 路由， z 又通过 y 路由。路由选择环路就像一个黑洞，即目的地为 x 的分组在 t_1 时刻到达 y 或 z 后，将在这两个结点之间不停地（或直到转发表发生改变为止）来回反复。

2) 因为结点 y 已算出到 x 的新的最低费用，它在 t_1 时刻将该新距离向量通知 z 。

3) 在 t_1 后某个时间, z 收到 y 的新距离向量, 它指示了 y 到 x 的新最低费用是 6。 z 知道它能以费用 1 到达 y , 因此计算出到 x 的新最低费用 $D_z(x) = \min\{50 + 0, 1 + 6\} = 7$ 。因为 z 到 x 的最低费用已增加了, 于是它便在 t_2 时刻通知 y 其新费用。

4) 以类似方式, 在收到 z 的新距离向量后, y 决定 $D_y(x) = 8$ 并向 z 发送其距离向量。接下来 z 确定 $D_z(x) = 9$ 并向 y 发送其距离向量, 等等。

该过程将要继续多久呢? 你应认识到循环将持续 44 次迭代 (在 y 与 z 之间交换报文), 即直到 z 最终算出它经由 y 的路径费用大于 50 为止。此时, z 将 (最终) 确定它到 x 的最低费用路径是经过它到 x 的直接连接。 y 将经由 z 路由选择到 x 。关于链路费用增加的坏消息的确传播得很慢! 如果链路费用 $c(y, x)$ 从 4 变为 10 000 且费用 $c(z, x)$ 为 9999 时将发生什么样的现象呢? 由于这种情况, 我们所见的问题有时被称为无穷计数 (count-to-infinity) 问题。

2. 距离向量算法: 增加毒性逆转

刚才描述的特定循环的场景可以通过使用一种称为毒性逆转 (poisoned reverse) 的技术而加以避免。其思想较为简单: 如果 z 通过 y 路由选择到目的地 x , 则 z 将通告 y , 它 (z) 到 x 的距离是无穷大, 即 z 将向 y 通告 $D_z(x) = \infty$ (即使 z 实际上知道 $D_z(x) = 5$)。只要 z 经 y 路由选择到 x , z 就持续地向 y 讲述这个善意的谎言。因为 y 相信 z 没有到 x 的路径, 故只要 z 继续经 y 路由选择到 x (并这样去撒谎), y 将永远不会试图经由 z 路由选择到 x 。

我们现在看一下毒性逆转如何解决我们前面在图 4-31b 中遇到的特定环路问题。作为毒性逆转的结果, y 的距离表指示了 $D_z(x) = \infty$ 。当 (x, y) 链路的费用在 t_0 时刻从 4 变为 60 时, y 更新其表, 虽然费用高达 60, 仍继续直接路由选择到 x , 并将到 x 的新费用通知 z , 即 $D_y(x) = 60$ 。 z 在 t_1 时刻收到更新后, 便立即将其到 x 的路由切换到经过费用为 50 的直接 (z, x) 链路。因为这是一条新的到 x 的最低费用路径, 且因为路径不再经过 y , z 就在 t_2 时刻通知 y 现在 $D_z(x) = 50$ 。在收到来自 z 的更新后, y 使用 $D_y(x) = 51$ 更新其距离表。另外, 因为 z 此时位于 y 到 x 的最低费用路径上, 所以 y 通过在 t_3 时刻通知 z 其 $D_y(x) = \infty$ (即使 y 实际上知道 $D_y(x) = 51$) 毒化从 z 到 x 的逆向路径。

毒性逆转解决了一般性的无穷计数问题吗? 没有。你应认识到涉及 3 个或更多结点 (而不只是两个直接相连的邻居结点) 的环路将无法用毒性逆转技术检测到。

3. LS 与 DV 路由选择算法的比较

DV 和 LS 算法采用互补的方法来解决路由选择计算问题。在 DV 算法中, 每个结点仅与它的直接相连的邻居交谈, 但它为其邻居提供了从它自己到网络中 (它所知道的) 所有其他结点的最低费用估计。在 LS 算法中, 每个结点 (经广播) 与所有其他结点交谈, 但它仅告诉它们与它直接相连链路的费用。我们通过快速比较它们各自的属性来总结所学的链路状态与距离向量算法。记住 N 是结点 (路由器) 的集合, 而 E 是边 (链路) 的集合。

- 报文复杂性。我们已经看到 LS 算法要求每个结点都知道网络中每条链路的费用。这就要求要发送 $O(|N||E|)$ 个报文。而且无论何时一条链路的费用改变时, 必须向所有结点发送新的链路费用。DV 算法要求在每次迭代时, 在两个直接相连邻居之间交换报文。我们已经看到, 算法收敛所需时间依赖于许多因素。当链路费用改变时, DV 算法仅当在新的链路费用导致与该链路相连结点的最低费用路径发生改变时, 才传播已改变的链路费用。

- 收敛速度。我们已经看到 LS 算法的实现是一个要求 $O(|N||E|)$ 个报文的 $O(|N|^2)$ 算法。DV 算法收敛较慢，且在收敛时会遇到路由选择环路。DV 算法还会遭遇无穷计数的问题。
- 健壮性。如果一台路由器发生故障、行为错乱或受到破坏时情况会怎样呢？对于 LS 算法，路由器能够向其连接的一条链路广播不正确费用（但是没有其他链路）。作为 LS 广播的一部分，一个结点也可损坏或丢弃它收到的任何 LS 广播分组。但是一个 LS 结点仅计算自己的转发表；其他结点也自行执行类似的计算。这就意味着在 LS 算法下，路由计算在某种程度上是分离的，提供了一定程度的健壮性。在 DV 算法下，一个结点可向任意或所有目的结点通告其不正确的最低费用路径。（在 1997 年，一个小 ISP 的一台有故障的路由器的确向美国的主干路由器提供了错误的路由选择信息。这引起了其他路由器将大量流量引向该故障路由器，并导致因特网的大部分中断连接达数小时 [Neumann 1997]。）更一般地，我们会注意到每次迭代时，在 DV 算法中一个结点的计算会传递给它的邻居，然后在下次迭代时再间接地传递给邻居的邻居。在此情况下，DV 算法中一个不正确的结点计算值会扩散到整个网络。

总之，两个算法没有一个是明显的赢家，它们的确都在因特网中得到了应用。

4. 其他路由选择算法

我们已学过的 LS 算法与 DV 算法不仅在实践中得到了广泛使用，而且它们基本上是当前因特网实践中使用的仅有的两种路由选择算法。无论如何，在过去的 30 年里，研究人员已提出了许多路由选择算法，从非常简单的到非常精致和复杂的都有。一种宽泛类型的路由选择算法是基于将分组流量看作是网络中源和目的之间的流。在这种方法中，路由选择问题可在数学上被形式化为一个称之为网络流问题的受限优化问题 [Bertsekas 1991]。然而，我们在此要提及的另外一类算法集合，是那些来源于电话界的路由选择算法。当每条链路资源（如缓冲区、链路带宽的一部分）需要保留给每条经过该链路的连接时，这些电路交换路由选择算法（circuit-switched routing algorithm）对分组交换数据网是很有价值的。虽然该路由选择问题的形式化可能看起来与我们在本章看到的最低费用路由选择的形式化十分不同，但也有许多相似性，至少对于路径查找算法（路由选择算法）是这样的。关于该研究领域的更详细的讨论，可参考 [Ash 1998; Ross 1995; Girard 1990]。

4.5.3 层次路由选择

在 LS 和 DV 算法的研究中，我们将网络只看作一个互联路由器的集合。从所有路由器执行相同的路由选择算法以计算穿越整个网络的路由选择路径的意义上来说，一台路由器很难同另一台路由器区别开来。在实践中，该模型和这种一组执行同样路由选择算法的同质路由器集合的观点有一点简单化，因为至少有以下两个重要原因：

- 规模。随着路由器数目变得很大，涉及路由选择信息的计算、存储及通信（例如 LS 更新或最低费用路径的变化）的开销将高得不可实现。当今的公共因特网由数亿台主机组成。在这些主机中存储的路由选择信息显然需要巨大容量的内存。在公共因特网上的所有路由器中广播 LS 更新所需的开销将导致没有剩余的带宽用来发送数据分组！在如此大量的路由器中迭代的距离向量算法将肯定永远无法收敛！显然，必须采取一些措施以减少公共因特网这种大网络中的路由选择计算的复杂性。

- 管理自治。虽然研究人员倾向于忽略这样的问题，如某公司要求按自己的意愿运行路由器（如运行其选择的某种路由选择算法），或对外部隐藏其网络的内部组织面貌，但这些都是需要考虑的重要因素。在理想情况下，一个组织应当能够按自己的愿望运行和管理其网络，还要能将其网络与其他外部网络相连接。

这两个问题都可以通过将路由器组织进自治系统（Autonomous System，AS）来解决，每个 AS 由一组通常处在相同管理控制下的路由器组成（例如，由相同的 ISP 运营或属于相同的公司网络）。在相同的 AS 中的路由器都全部运行同样的路由选择算法（如一种 LS 或 DV 算法），且拥有彼此的信息，这就像在前一节中所讲的理想化模型中的情况一样。在一个自治系统内运行的路由选择算法叫做自治系统内部路由选择协议（intra-autonomous system routing protocol）。当然，将 AS 彼此互联是必需的，因此在一个 AS 内的一台或多台路由器将有另外的任务，即负责向在本 AS 之外的目的地转发分组。这些路由器被称为网关路由器（gateway router）。

图 4-32 提供了具有 3 个 AS 的简单例子：AS1、AS2 和 AS3。在该图中，粗线表示了路由器对之间的直接链路连接。从路由器连出的细线表示直接与路由器连接的子网。AS1 具有 4 台路由器，1a、1b、1c 和 1d，它们运行在 AS1 内部，使用了 AS1 内部路由选择协议。因此，这 4 台路由器的每一台都知道如何沿着优化路径转发到 AS1 内任何目的地的分组。类似地，自治系统 AS2 和 AS3 每个都有 3 台路由器。注意到运行在 AS1、AS2 和 AS3 中的 AS 内部路由选择不必是相同的。同时注意到路由器 1b、1c、2a 和 3a 都是网关路由器。

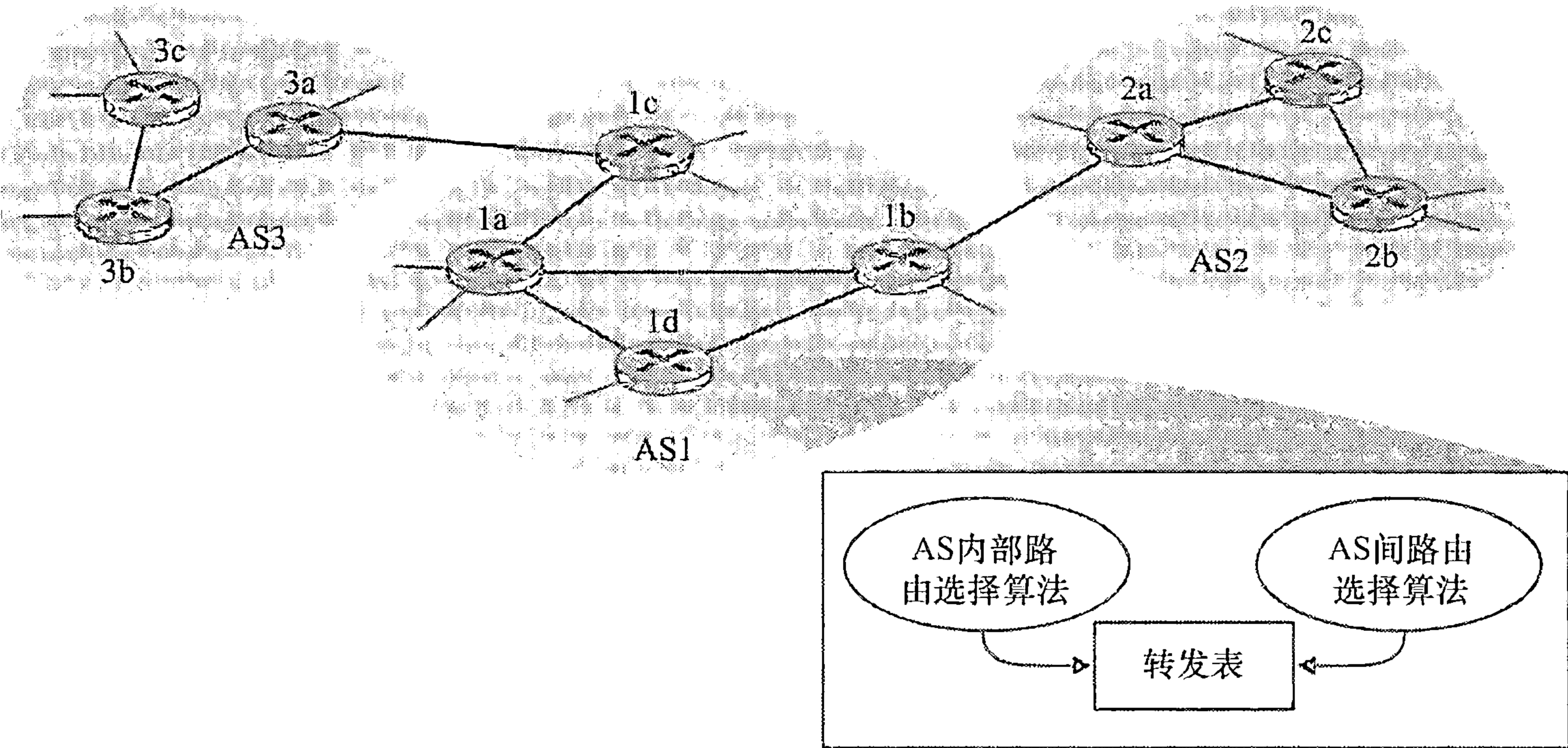


图 4-32 一个互联的自治系统的例子

现在应当清楚在一个 AS 中的路由器是怎样为在 AS 内部的源和目的对决定路由选择路径的。但是对于端到端路由选择难题仍然有一大块遗漏的部分，即在某些 AS 中一台路由器，怎样知道该如何将分组路由选择到位于该 AS 外部的目的地呢？如果 AS 仅有一个网关路由器连接唯一一个其他 AS 的话，回答这个问题是容易的。在这种情况下，因为该 AS 内部的 AS 路由选择协议已经决定了从内部路由器到网关路由器的最低费用路径，因此每台内部路由器知道它应当如何转发分组。网关路由器一旦接收到分组，将分组向通向外部的 AS 的一条链路转发。该链路另一端的 AS 承担起将该分组向其最终目的地路由选择的责任。举一个例子，假定在图 4-32 中的路由器 2b 接收到一个分组，其目的地位于 AS2 的

外部。路由器 2b 则将该分组转发到路由器 2a 或 2c，这由路由器 2b 的转发表指定，该转发表由 AS2 的 AS 内部路由选择协议所配置。该分组最后到达网关路由器 2a，2a 又将分组转发到 1b。一旦该分组离开 2a，AS2 对这个分组的任务就完成了。

因此当源 AS 仅有一条通向外部 AS 的链路时，容易解决该问题。如果源 AS 具有两条或更多条链路（通过一台或更多台网关路由器）通向外部 AS，情况将会怎样呢？此时向何处转发分组的问题变得更具挑战性。例如，考虑在 AS1 中的一台路由器，假定它接收了一个目的地在该 AS 外部的分组。路由器显然应当向它的两个网关路由器之一（1b 或 1c）转发该分组，但到底是哪个呢？为了解决这个问题，AS1 需要：①知道经 AS2 可达哪些目的地，经 AS3 可达哪些目的地；②向 AS1 中的所有路由器传播这些可达性信息，因此每台路由器能够配置它的转发表以处理外部 AS 目的地。从相邻 AS 获取可达性信息和向该 AS 中所有路由器传播可达性信息是两项由自治系统间路由选择协议（inter-autonomous system routing protocol）处理的任务。因为自治系统间路由选择协议涉及两个 AS 之间的通信，这两个通信的 AS 必须运行相同的自治系统间路由选择协议。事实上，因特网中的所有 AS 中都运行相同的 AS 间路由选择协议，该协议称为 BGP4，我们将在下一节中讨论。如图 4-32 所示，每台路由器接收来自一个 AS 内部路由选择协议和一个 AS 间路由选择协议的信息，并使用来自这两个协议的信息配置它的转发表。

举一个例子，考虑子网 x （用它的 CDIR 化的地址标识），假定 AS1 从 AS 间路由选择协议知道子网 x 从 AS3 可达，而从 AS2 不可达。AS1 则向它的所有路由器传播这个信息。当路由器 1d 知道从 AS3 并因此从网关 1c 可达子网 x 时，根据 AS 内部路由选择协议提供的信息，它进而确定该路由器接口位于从路由器 1d 到网关路由器 1c 的最低费用路径上，比如这是接口 I 。路由器 1d 则能够将表项 (x, I) 放入其转发表中。（这个例子和在本节中给出的其他例子，体现了因特网中实际发生情况的一般但被简化的概念。在下一节中，当我们讨论 BGP 时，将提供一个更为详细的描述，尽管它更为复杂。）

接着上面的例子，现在假设 AS2 和 AS3 与其他 AS 相连，这些 AS 并没有在该图上显示出来。同时假定 AS1 从 AS 间路由选择协议知道了子网 x 是可达的：或者从 AS2 经网关 1b，或者从 AS3 经网关 1c。AS1 则将向它的所有路由器（包括路由器 1d）传播该信息。为了配置它的转发表，路由器 1d 将必须决定通过 1b 或 1c 网关路由器中的哪个来指引目的地为子网 x 的分组。在实践中经常使用的一种方法是热土豆路由选择（hot potato routing）。在热土豆路由选择中，AS 尽可能快地（更准确地讲是尽可能经济地）扔掉分组（热土豆）。这通过让路由器向某网关路由器发送分组来完成，同时该网关路由器在到目的地路径上的所有网关路由器中有最低的路由器到网关的费用。在当前例子环境中，在 1d 中运行的热土豆路由选择将使用来自 AS 内部路由选择协议的信息，以决定到 1b 和 1c 的路径费用，并选择具有最低费用的路径。一旦选择这条路径，路由器 1d 在其转发表中增加用于子网 x 的表项。图 4-33 总结了路由器 1d 对转发表增加用于 x 的表项所采取的动作。

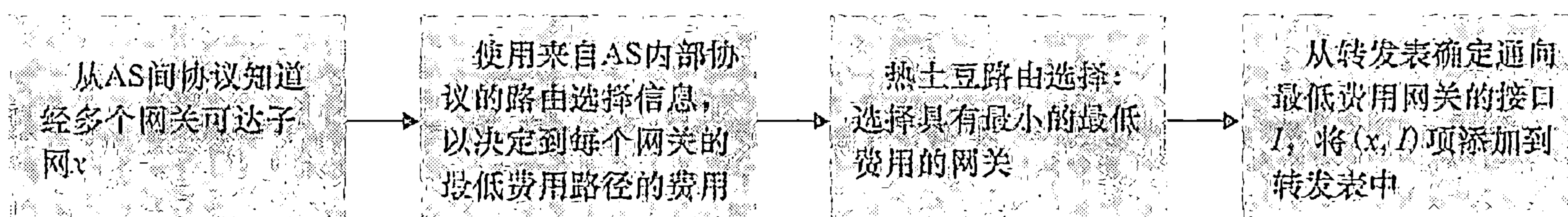


图 4-33 在路由器的转发表中增加一个 AS 之外目的地的步骤

当一个 AS 从一个相邻 AS 处知道了一个目的地，该 AS 能够向它的某些其他相邻 AS 通告该路由选择信息。例如，假定 AS1 从 AS2 处知道了经 AS2 能到达子网 x 。AS1 则能够告诉 AS3 经 AS1 能到达 x 。以这种方式，如果 AS3 需要为一个目的地是 x 的分组路由的话，AS3 将向 AS1 转发该分组，AS1 将依次向 AS2 转发该分组。如我们将在讨论 BGP 时所见，一个 AS 在决定向其相邻 AS 通告哪些目的地时，具有相当大的灵活性。这是一个策略决定，通常更多地取决于经济问题而不是技术问题。

1.5 节讲过，因特网是由等级制的互联 ISP 组成。那么 ISP 和 AS 之间是什么样的关系呢？你也许认为在一个 ISP 中的路由器和互联它们的链路构成了单个 AS。虽然经常是这种情况，但许多 ISP 将它们的网络划分为多个 AS。例如，某些第一层 ISP 对它们的整个网络使用一个 AS；其他 ISP 则将它们的 ISP 划分成数十个互联的 AS。

总而言之，规模与管理责任的问题可通过定义自治系统来解决。在一个 AS 内部，所有路由器运行同样的自治系统内部路由选择协议。在各 AS 之间，AS 运行相同的 AS 间路由选择协议。因为一个 AS 内部路由器仅需要知道本 AS 内的路由器，扩展性问题得到了解决。因为一个组织可运行它选择的任何 AS 内部路由选择协议，管理责任的问题得到了解决；然而，相连的每对 AS 需要运行相同的 AS 间路由选择协议以交换可达性信息。

在下一节中，我们将学习当今因特网中使用的两种 AS 内部路由选择协议（RIP 与 OSPF）和一种 AS 间路由选择协议（BGP）。这些学习案例将很好地从不同侧面帮助我们学习层次路由选择内容。

4.6 因特网中的路由选择

学习了因特网编址与 IP 协议以后，我们现在将注意力转到因特网路由选择协议上来。路由选择协议的任务就是要确定数据报在源与目的地之间采用的路径。我们将看到，因特网的路由选择协议包含了许多我们在本章前面学过的原理。在 4.5.1 节和 4.5.2 节中学过的链路状态与距离向量方法，以及在 4.5.3 节中考虑的自治系统概念，这些都在当今的因特网的路由选择过程中起到至关重要的作用。

在 4.5.3 节中讲过，一个 AS 是一个处于相同的管理与技术控制下的路由器的集合，在 AS 之间都运行相同的路由选择协议。每个 AS 通常又都包含多个子网（这里使用的术语子网具有 4.4.2 节中的精确、编址的含义）。

4.6.1 因特网中自治系统内部的路由选择：RIP

AS 内部路由选择协议用于确定在一个 AS 内执行路由选择的方式。AS 内部路由选择协议又称为内部网关协议（interior gateway protocol）。历史上有两个路由选择协议曾被广泛用于因特网上自治系统内的路由选择：路由选择信息协议（Routing Information Protocol, RIP）与开放最短路径优先（Open Shortest Path First, OSPF）。与 OSPF 密切相关的路由选择协议是 IS-IS 协议 [RFC 1142; Perlman 1999]。我们首先讨论 RIP，然后考虑 OSPF。

RIP 是最早的 AS 内部因特网路由选择协议之一，且目前仍在广泛使用。它的产生与命名源于 Xerox 网络系统（XNS）体系结构。RIP 的广泛应用主要是由于它被包含在支持 TCP/IP 的 1982 年的 UNIX 伯克利软件分发（BSD）版本中。在 [RFC 1058] 中定义了 RIP 版本 1，在 [RFC 2453] 中定义了其向后兼容的版本 2。

RIP 是一种距离向量协议，其运行方式很像我们在 4.5.2 节中学习的理想化 DV 协议。在 RFC 1058 中定义的 RIP 版本使用跳数作为其费用测度，即每条链路的花费为 1。在 4.5.2 节的 DV 算法中，为了简单起见，费用被定义在路由器对之间。在 RIP（也在 OSPF）中，费用实际上是从源路由器到目的子网。RIP 使用术语跳，跳是沿着从源路由器到目的子网（包括目的子网）的最短路径所经过的子网数量。图 4-34 举例说明了一个具有 6 个叶子子网的 AS。在该图中的表指出了从源 A 到每个叶子子网的跳数。

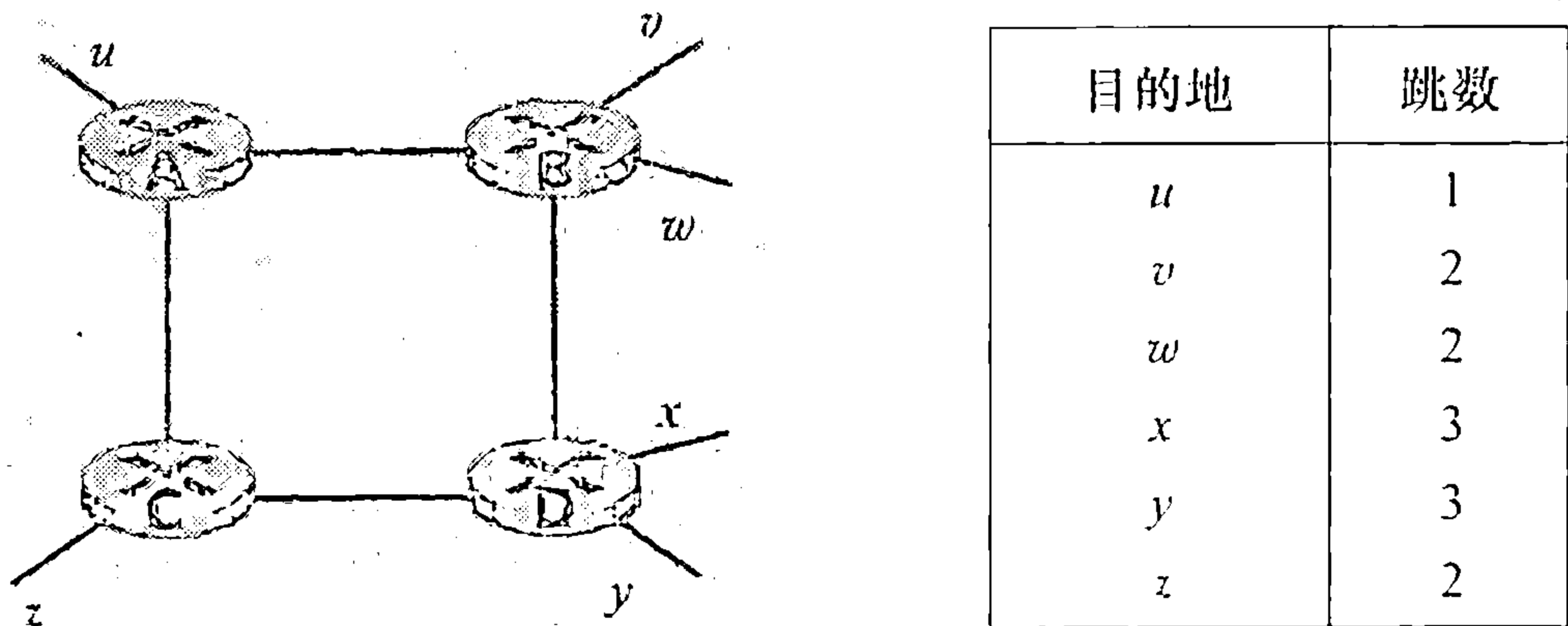


图 4-34 从源路由器 A 到各个子网的跳数

一条路径的最大费用被限制为 15，因此 RIP 的使用限制在网络直径不超过 15 跳的自治系统内。前面讲过在 DV 协议中，相邻路由器之间相互交换路由选择信息。任何一台路由器的距离向量是从这台路由器到该 AS 中子网的最短路径距离的当前估计值。在 RIP 中，路由选择更新信息在邻居之间通过使用一种 RIP 响应报文（RIP response message）来交换，大约每 30 秒相互交换一次。由一台路由器或主机发出的响应报文包含了一个该 AS 内的多达 25 个目的子网的列表，以及发送方到其中每个子网的距离。响应报文又被称作 RIP 通告（RIP advertisement）。

我们考察有关 RIP 通告是如何工作的一个简单例子。考虑在图 4-35 中显示的一个 AS 的一部分。在该图中，连接路由器的线表示子网。仅有选中的路由器（A、B、C、D）与网络（*w*、*x*、*y*、*z*）被标记了。虚线指示了该 AS 还在继续，因此该自治系统有比图示更多的路由器和链路。

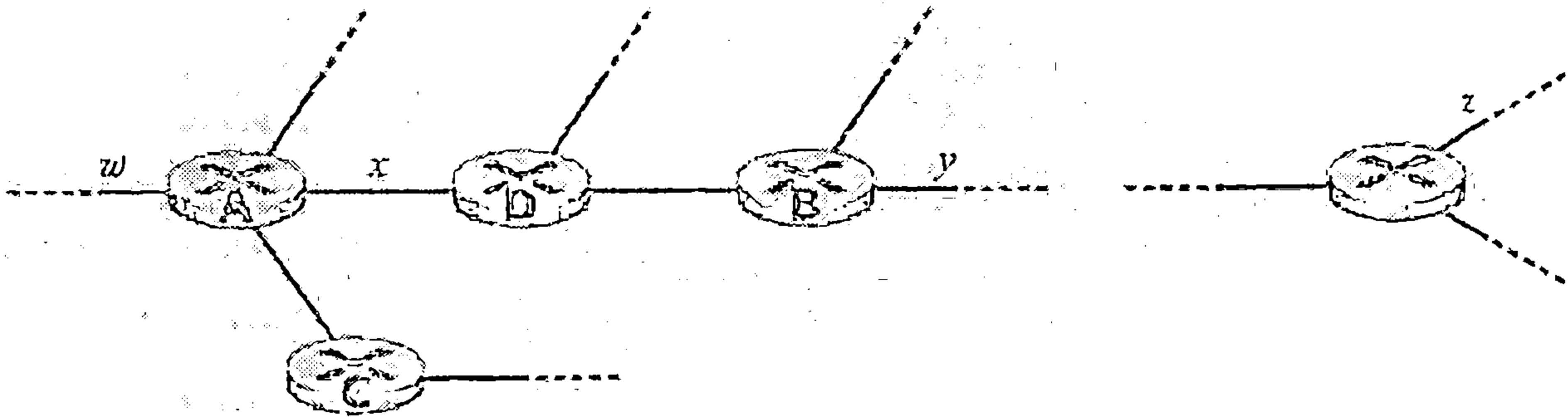


图 4-35 一个自治系统的一部分

每台路由器维护一张称为路由选择表（routing table）的 RIP 表。一台路由器的路由选择表包括该路由器的距离向量和该路由器的转发表。图 4-36 显示了路由器 D 的转发表。注意到该转发表有 3 列。第一列用于目的子网，第二列指示了沿着最短路径到目的子网的下一个路由器的标识，第三列指出了沿着最短路径到目的子网的跳数（即需要穿越的子网数，包括目的子网）。对于该例而言，该表指出了从路由器 D 到目的子网 *w* 发送一个数据

报，该数据报应首先被转发到相邻路由器 A；该表还指出沿着最短路径到目的子网 w 为两跳距离。类似地，该表指出了子网 z 经由路由器 B 为 7 跳距离。虽然 RIP 版本 2 允许使用类似于我们在 4.4 节中学习的路由聚合技术来聚合子网表项，但原则上 AS 内的每个子网应在转发表中占一行。在图 4-36 中的表以及后续的表，都只是部分完成了。

目的子网	下一台路由器	到目的地的跳数
w	A	2
y	B	2
z	B	7
x	—	1
...	...	...

图 4-36 收到来自路由器 A 的通告之前路由器 D 中的转发表

现在假定 30 秒以后，路由器 D 收到来自路由器 A 的如图 4-37 所示的通告。注意到该通告正是来自路由器 A 的路由选择表信息！该信息特别指明了子网 z 离路由器 A 仅有 4 跳距离。一旦收到该通告，路由器 D 将该通告（图 4-37）与旧路由选择表（图 4-36）合并。特别是路由器 D 知道了通过路由器 A 到子网 z 比通过路由器 B 到达路径更短。因此，路由器 D 更新其转发表以记下该更短的最短路径，如图 4-38 中所示。你也许会问，到子网 z 的最短路径怎么会变得更短呢？可能是分布式的距离向量算法还处在收敛过程中（参见 4.5.2 节），或者也许是新的链路和/或路由器加入了该 AS，因此改变了在 AS 中的最短路径。

目的子网	下一台路由器	到目的地的跳数
z	C	4
w	—	1
x	—	1
...	...	...

图 4-37 来自路由器 A 的通告

目的子网	下一台路由器	到目的地的跳数
w	A	2
y	B	2
z	A	5
...	...	...

图 4-38 收到路由器 A 的通告后路由器 D 中的转发表

我们下面考虑 RIP 实现方面的几个问题。前面讲过 RIP 路由器大约每 30 秒相互交互通告。如果一台路由器一旦超过 180 秒没有从邻居听到报文，则该邻居不再被认为是可达的；即要么其邻居死机了，要么连接的链路中断了。当这种情况发生时，RIP 修改本地路由选择表，然后通过向相邻路由器（那些仍然可达的路由器）发送通告来传播该信息。路由器也可通过使用 RIP 请求报文，请求其邻居到指定目的地的费用。路由器在 UDP 上使用端口 520 相互发送 RIP 请求与响应报文。封装在标准 IP 数据报中的 UDP 报文段在路由器之间传输。RIP 使用一个位于网络层协议（IP）之上的运输层协议（UDP）来实现网络层功能（一种路由选择算法），这个事实看起来似乎相当令人费解（事实如此!）。若再深入一些观察 RIP 的实现原理将能更明白这一点。

图 4-39 概略地说明了 RIP 在一个 UNIX 系统中通常是如何实现的，例如一台用作路由器的 UNIX 工作站。一个称为 routed 的进程执行 RIP，即维护路由选择信息并与相邻路由器中的 routed 进程交换报文。因为 RIP 是被当作一个应用层进程来实现的（虽然它是一个能操作 UNIX 内核中的转发表的特殊进程），它能在一个标准套接字上发送和接收报文，并且使用一个标准的运输层协议。如显示的那样，RIP 是一个运行在 UDP 上的应用层协议（参见第 2 章）。如果读者有意关注 RIP（或我们将很快学习的 OSPF 和 BGP 协议）的实现，请参阅 [Quagga 2012]。

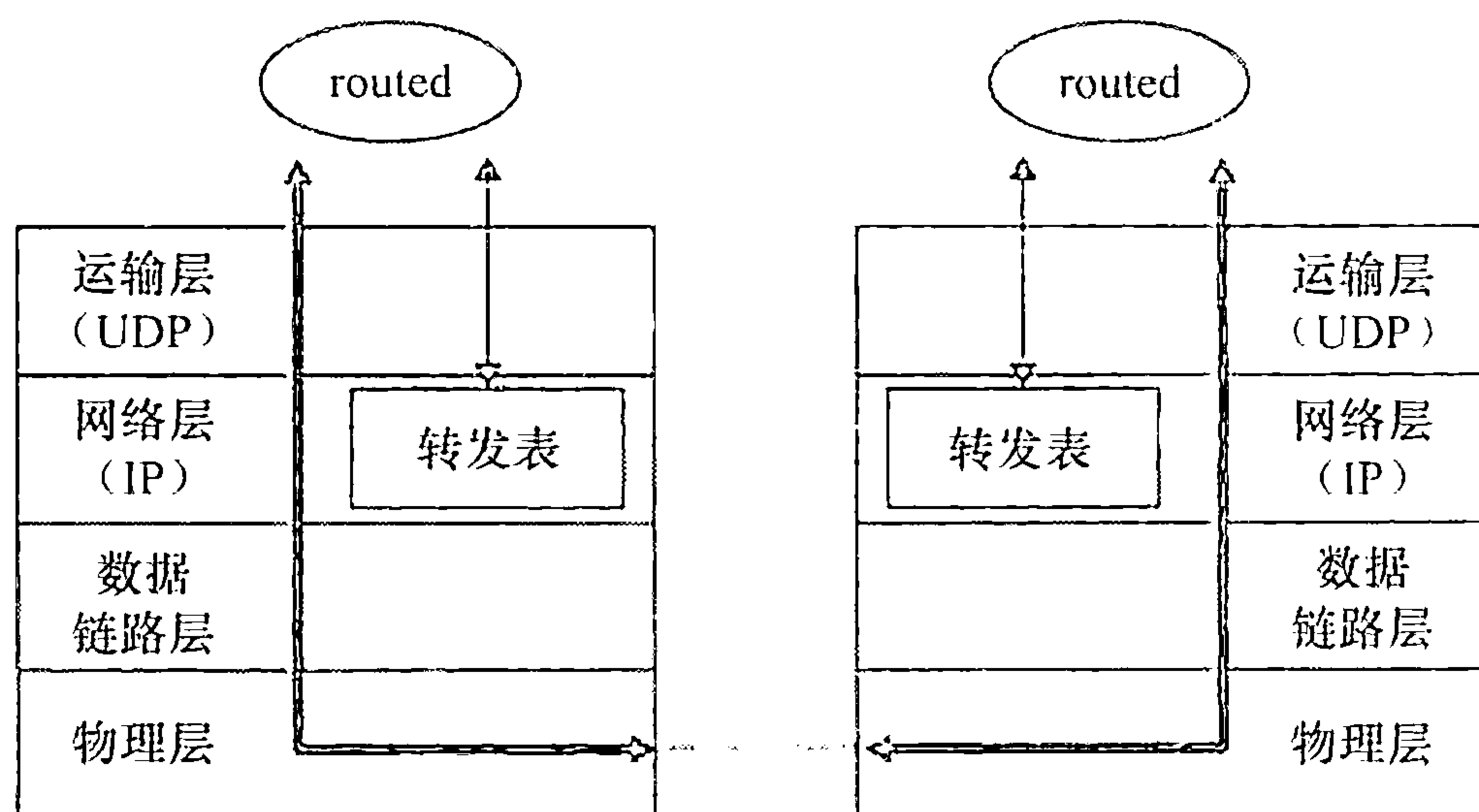


图 4-39 作为 routed 守护程序的 RIP 实现

4.6.2 因特网中自治系统内部的路由选择：OSPF

就像 RIP 一样，OSPF 路由选择也被广泛用于因特网的 AS 内部路由选择。OSPF 和它的关系密切的表兄弟 IS-IS 通常都设置在上层的 ISP 中，而 RIP 却被设置在下层 ISP 和企业网中。OSPF 中的开放（Open）一词是指路由选择协议规范是公众可用的（与之相反的是如 Cisco 的 EIGRP 协议）。OSPF 的最新版本是版本 2，由 RFC 2328 这个公用文档所定义。

OSPF 被设想为是 RIP 的后继者，因为它有许多先进特性。然而，OSPF 的核心就是一个使用洪泛链路状态信息的链路状态协议和一个 Dijkstra 最低费用路径算法。使用 OSPF，一台路由器构建了一幅关于整个自治系统的完整拓扑图（即一个图）。于是，路由器在本地运行 Dijkstra 的最短路径算法，以确定一个以自身为根结点的到所有子网的最短路径树。各条链路费用是由网络管理员配置的（参见“实践原则：设置 OSPF 链路权值”）。管理员也许会选择将所有链路费用设为 1，因而实现了最少跳数路由选择，或者可能会选择将链路权值按与链路容量成反比来设置，从而不鼓励流量使用低带宽链路。OSPF 不强制使用如何设置链路权值的策略（那是网络管理员的任务），但提供了一种机制（协议），为给定链路权值集合确定最低费用路径路由选择。

使用 OSPF 时，路由器向自治系统内所有其他路由器广播路由选择信息，而不仅仅是向其相邻路由器广播。每当一条链路的状态发生变化时（如费用的变化或连接/中断状态的变化），路由器就会广播链路状态信息。即使链路状态未发生变化，它也要周期性地（至少每隔 30 分钟一次）广播链路状态。RFC 2328 中有这样的说明：“链路状态通告的这种周期性的更新增加了链路状态算法的健壮性。”OSPF 通告包含在 OSPF 报文中，该 OSPF 报文直接由 IP 承载，对 OSPF 其上层协议的值为 89。因此 OSPF 协议必须自己实现诸如可靠报文传输、链路状态广播等功能。OSPF 协议还要检查链路正在运行（通过向相连的邻居发送 HELLO 报文），并允许 OSPF 路由器获得相邻路由器的网络范围链路状态的数据库。

OSPF 的优点包括下列几方面：

- 安全。能够鉴别 OSPF 路由器之间的交换（如链路状态更新）。使用鉴别，仅有受信任的路由器能参与一个 AS 内的 OSPF 协议，因此可防止恶意入侵者（或正在利用新学的知识到处试探的网络专业的学生）将不正确的信息注入路由器表内。在默认状态下，路由器间的 OSPF 报文是未被鉴别的并能被伪造。能够配置两类鉴别，即简单的和 MD5 的（参见第 8 章有关 MD5 和鉴别的一般性讨论）。使用简单